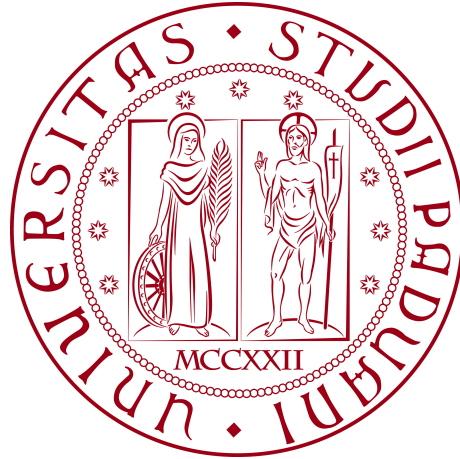


Università degli Studi di Padova

DIPARTIMENTO DI MATEMATICA “TULLIO LEVI-CIVITA”

CORSO DI LAUREA IN INFORMATICA



Progettazione di un Motore di Previsione

Vendite per Sistemi ERP Aziendali

Tesi di Laurea Triennale

Relatrice

Prof.ssa Losiouk Eleonora

Laureando

Oliver Florin Stiglet

Matricola 2044895

ANNO ACCADEMICO 2025-2026

“It’s not a bug, it’s a feature.”

— Matt Groening, *Futurama*.

Ringraziamenti

Desidero esprimere la mia gratitudine alla Prof.ssa Losiouk Eleonora, mia relatrice, per l’aiuto e il sostegno che mi ha dato durante la stesura dell’elaborato.

Vorrei anche ringraziare, con affetto, i miei genitori per il loro sostegno, il grande aiuto e la loro presenza in ogni momento durante gli anni di studio.

Desidero poi ringraziare i miei amici per i bellissimi anni trascorsi insieme e le mille avventure vissute.

Padova, Dicembre 2025

Oliver Florin Stiglet

Sommario

Durante lo stage presso Spazio Dev ho contribuito alla realizzazione di *ErpAI*, un nuovo ERP modulare. Il lavoro si è concentrato sull'implementazione di un motore di previsione vendite per anticipare la produzione: a partire da uno storico di fatture che descrivono le vendite passate dei prodotti, il modello stima la domanda futura per supportare decisioni operative tempestive e ridurre il rischio di stock-out. La tesi ripercorre l'analisi dei requisiti, le scelte architetturali e tecnologiche, l'implementazione del servizio esposto via API, la strategia di testing e integrazione continua, fino alla valutazione dei risultati ottenuti. La tesi documenta progettazione, scelta delle feature e addestramento del modello XGBoost, selezionato per l'equilibrio tra accuratezza predittiva, tempi di training e impiego in contesti produttivi.

Indice

Acronimi e abbreviazioni	xiii
Glossario	xv
1 Introduzione	2
1.1 L'azienda	2
1.1.1 ErpAI	2
1.2 L'idea	3
1.3 Organizzazione del testo	3
2 Processi aziendali	5
2.1 Organizzazione e collaborazione	5
2.1.1 Ambienti e strumenti di sviluppo	5
2.1.2 Modello di lavoro agile	6
2.1.3 Comunicazione e documentazione	6
2.2 Strumenti tecnici e controllo qualità	7
2.2.1 Version control e branching	7
2.2.2 Issue tracking	8
3 Descrizione dello stage	10
3.1 Introduzione al progetto	10
3.2 Analisi preventiva dei rischi	10
3.3 Pianificazione	11
3.3.1 Modalità di stage e interazione	11
3.3.2 Pianificazione settimanale	12

4	Analisi dei requisiti	13
4.1	Casi d'uso	13
4.1.1	Attori individuati	13
4.1.2	Casi d'uso	14
4.1.3	Diagrammi dei casi d'uso	17
4.2	Requisiti	19
4.2.1	Codifica requisiti	19
4.2.2	Elenco requisiti	19
5	Progettazione	22
5.1	Tecnologie e strumenti	22
5.1.1	Linguaggi di programmazione	22
5.1.2	Framework e server web	23
5.1.3	Database e ORM	23
5.1.4	Autenticazione e sicurezza	24
5.1.5	Machine Learning e Data Science	24
5.1.6	Strumenti di testing e deployment	25
5.2	Architettura	25
5.2.1	API Gateway	27
5.2.2	Auth Server	27
5.2.3	Modulo DuckDB	27
5.2.4	Modulo di Previsione	28
5.2.5	Prevision DB	29
5.2.6	Chroma DB	30
5.2.7	Auth DB	30
5.3	Design pattern adottati	31
5.3.1	Architettura a layer	31
5.3.2	Repository pattern	31
5.3.3	Inversione del controllo	33
6	Codifica	34
6.1	Struttura del codice	34
6.1.1	Root	34

6.1.2	forecasting	35
6.2	Modelli dei dati	37
6.2.1	Modelli Pydantic	37
6.2.1.1	Article	37
6.2.1.2	Invoice	37
6.2.1.3	Job, JobKind, JobStatus	38
6.2.1.4	NotificationEvent	39
6.2.1.5	ChromaCollection	40
6.2.1.6	ShapExplanation	41
6.2.1.7	PredictionResult	41
6.2.1.8	ForecastSearchResult	42
6.2.2	Modelli SQLModel	42
6.2.2.1	XGBoostPrediction	42
6.2.2.2	ForecastRun	43
6.2.2.3	JobRun (SQLModel)	44
6.2.3	Modelli di configurazione	46
6.2.3.1	Modello di configurazione JWTConfig	46
6.2.3.2	Modello di configurazione XGBoostConfig	46
6.3	Servizi	48
6.3.1	InvoiceConversionService	49
6.3.2	DataFrameService	51
6.3.3	ForecastingDataService	54
6.3.4	MetricsService	56
6.3.5	NotificationService	57
6.3.6	DataNormalizeService	59
6.3.7	XgboostService	59
6.3.7.1	Inizializzazione	60
6.3.7.1.1	Training.	60
6.3.7.2	Previsione	62
6.3.8	ShapService	63
6.3.9	ForecastingService	65
6.4	API Router	68

6.4.1	Authentication (Auth Server)	68
6.4.2	/verify-token	69
6.4.3	/forecast	69
6.4.4	/jobs	72
6.4.5	/context	72
7	Testing e Continuous Integration	75
7.1	Unit Test	75
7.1.1	Esempio	75
7.2	Integration Tests	80
7.2.1	Esempio	80
7.3	Continuous Integration	83
8	Conclusioni	87
8.1	Copertura dei requisiti	87
8.2	Analisi del prodotto	88
8.2.1	Previsioni annuali	88
8.2.2	Feature più influenti	89
8.2.3	Miglioramenti futuri	91
8.3	Valutazione personale	92

Elenco delle figure

1.1	Logo di Spazio Dev	2
2.1	Schema semplificato del <i>framework</i> Scrum adottato	6
2.2	Schema del branching model adottato	8
2.3	Schema semplificato della board Kanban su Plane	9
4.1	UC.1: avvio e invio richiesta di previsione	17
4.2	UC.2: inoltro della richiesta da <i>ErpAI</i> al modulo di previsione .	18
4.3	UC.3: elaborazione e notifica da parte del modulo di previsione	18
4.4	UC.4: interazione del modulo chatbot con il modulo di previsione	19
5.1	Architettura complessiva del modulo <i>ErpAI-Forecasting</i>	26
6.1	Struttura delle directory del progetto.	36
6.2	Panoramica dei servizi e delle loro interazioni.	49
8.1	Andamento storico vs previsioni annuali (screenshot della dash- board)	89
8.2	Tabella di dettaglio delle previsioni annuali	89

8.3	Feature più influenti secondo SHAP (valori e segno del contributo)	91
-----	--	----

Elenco delle tabelle

3.1	Pianificazione settimanale delle attività di stage.	12
4.1	Tabella dei casi d'uso	14
4.2	Tabella dei requisiti	19
8.1	Tabella di copertura dei requisiti	87

Elenco dei codici sorgenti

5.1	Esempio di <i>repository</i> per Chroma.	32
5.2	Esempio di dependency injection.	33
6.1	Modello Pydantic <code>Article</code>	37
6.2	Modello Pydantic <code>Invoice</code>	38
6.3	Modelli Pydantic <code>Job/JobKind/JobStatus</code>	39
6.4	Modello Pydantic <code>NotificationEvent</code>	40
6.5	Modello Pydantic <code>ChromaCollection</code>	40
6.6	Modello Pydantic <code>ShapExplanation</code>	41
6.7	Modello Pydantic <code>PredictionResult</code>	41
6.8	Modello Pydantic <code>ForecastSearchResult</code>	42
6.9	Modello <code>SQLModel XGBoostPrediction</code>	43
6.10	Modello <code>SQLModel ForecastRun</code>	44
6.11	Modello <code>SQLModel JobRun</code>	46
6.12	Dataclass <code>JWTConfig</code>	46
6.13	Dataclass di configurazione <code>XGBoostConfig</code>	48
6.14	Servizio <code>InvoiceConversionService</code>	51
6.15	Servizio <code>DataFrameService</code>	54
6.16	Servizio <code>ForecastingDataService</code>	56
6.17	Servizio <code>MetricsService</code>	57
6.18	Servizio <code>NotificationService</code>	58
6.19	Inizializzazione di <code>XGBoostService</code>	60
6.20	Metodo <code>train_model</code> di <code>XGBoostService</code>	62
6.21	Metodo <code>predict</code> di <code>XGBoostService</code>	63
6.22	Metodo <code>explain_prediction</code> di <code>ShapService</code>	64

6.23 Metodo <code>generate_summary_plot</code> di <code>ShapService</code>	65
6.24 Metodo <code>forecast</code> di <code>ForecastingService</code>	67
6.25 Router di autenticazione: <code>/.well-known/jwks.json</code> , <code>/token</code> (modulo <code>auth-server</code>).	68
6.26 Router <code>/verify-token</code>	69
6.27 Router <code>/forecast</code>	72
6.28 Router <code>/jobs</code>	72
6.29 Router <code>/context</code>	74
7.1 Unit test del servizio di forecasting.	80
7.2 Integration test per l'endpoint <code>/context</code>	83
7.3 Pipeline di Continuous Integration	86

Acronimi e abbreviazioni

CRUD Create, Read, Update, Delete. [31](#)

CSV Comma-Separated Values. [21](#)

GPU Graphics Processing Unit. [25](#)

HTML HyperText Markup Language. [84](#)

HTTP Hypertext Transfer Protocol. [29](#), [57](#)

IDE Integrated Development Environment. [5](#)

JSON JavaScript Object Notation. [21](#)

JWKS JSON Web Key Set. [20](#), [27](#), [68](#)

JWT JSON Web Token. [24](#), [26](#), [27](#), [46](#), [57](#), [68](#), [69](#), [80](#)

MAPE Mean Absolute Percentage Error. [28](#), [41](#), [56](#), [62](#), [65](#)

MRP Material Requirements Planning. [10](#)

REST Representational State Transfer. [7](#), [26](#), [28](#), [31](#), [35](#), [68](#)

RMSE Root Mean Squared Error. [28](#), [47](#)

SHAP SHapley Additive exPlanations. [3](#), [25](#), [29](#), [30](#), [36](#), [41](#), [42](#), [59](#), [63](#), [65](#), [72](#)

SMAPE Symmetric Mean Absolute Percentage Error. [56](#)

UUID Universally Unique Identifier. [42](#), [44](#)

WAPE Weighted Absolute Percentage Error. [28](#), [56](#), [60](#), [88](#)

XML eXtensible Markup Language. [84](#)

Glossario

API Acronimo di Application Program Interface: interfaccia applicativa che espone funzionalità e dati di un sistema software tramite contratti stabili; consente a componenti esterni di invocare operazioni e scambiare payload strutturati senza conoscere l'implementazione interna. Include aspetti di versioning, sicurezza e documentazione per garantire integrazioni affidabili.. [3](#), [4](#), [7](#), [11](#), [13](#), [14](#), [16](#), [20–27](#), [31](#), [35](#), [37](#), [65](#), [80](#)

ASGI Acronimo di Asynchronous Server Gateway Interface: specifica di interfaccia per applicazioni web asincrone che definisce un contratto standard tra server e applicazioni, abilitando runtime basati su event-loop, connessioni persistenti e gestione efficiente di I/O concorrente; separa lo stack di trasporto dal codice applicativo e consente server/worker intercambiabili.. [23](#)

Backlog Elenco prioritizzato di attività, requisiti o user story da eseguire, gestito e raffinato nel tempo dal team di prodotto. [6](#), [8](#), [19](#)

Modello di branching Strategia di gestione dei rami Git (es. main/development/feature/fix/docs) che definisce come organizzare sviluppo, integrazione e rilascio. [7](#)

Callback / Webhook Chiamata HTTP inviata da un sistema a un endpoint esterno per notificare eventi o risultati, spesso firmata/autenticata e usata per integrazioni push. [13](#), [21](#), [29](#), [39](#), [57](#)

DevOps Insieme di pratiche e cultura collaborativa che integra sviluppo (Dev) e operations (Ops) per automatizzare build, test, rilascio, monitoraggio e feedback continuo; punta a cicli di consegna rapidi e affidabili. [92](#)

DTO Data Transfer Object: oggetto leggero e tipizzato usato per trasportare dati tra livelli o servizi, privo di logica di dominio; riduce l'accoppiamento e rende espliciti contratti di input/output. [31](#)

Embedding Rappresentazione vettoriale densa di testo o altri oggetti che preserva relazioni semantiche, usata per similarity search e modelli downstream. [23](#), [30](#), [40](#), [59](#), [72](#)

ERP Acronimo di Enterprise Resource Planning: sistema informativo integrato che centralizza anagrafiche, processi amministrativi, logistici e produttivi in un unico database e set di moduli; garantisce coerenza dei dati, tracciabilità delle operazioni e automazione di workflow aziendali.. [2](#), [3](#), [5](#), [7](#), [11](#), [20](#), [26](#), [27](#), [34](#)

Feature engineering Processo di creazione, trasformazione e selezione di variabili derivate dai dati grezzi per migliorare l'addestramento e le prestazioni dei modelli. [28](#), [54](#)

Happy path Percorso di esecuzione ideale senza errori o eccezioni, usato nei test per validare il comportamento atteso in condizioni nominali. [76](#)

Hold-out Sottinsieme dei dati tenuto da parte rispetto al training per valutare in modo indipendente le prestazioni di un modello. [60](#)

Job / Run di previsione Esecuzione identificata (spesso con UUID) che traccia uno specifico ciclo di elaborazione o previsione, con risultati, metriche e log associati. [42](#), [44](#)

Kanban board Lavagna organizzata in colonne (es. Backlog, Todo, In Progress, Done) per visualizzare e limitare il lavoro in corso secondo i principi Kanban. [8](#)

LLM Acronimo di Large Language Model: modello di intelligenza artificiale di grandi dimensioni addestrato su corpora estesi per generare, riassumere e comprendere linguaggio naturale; utilizza architetture neurali con molti parametri per produrre risposte contestuali e coerenti in numerosi domini applicativi.. [24](#), [27](#), [30](#), [59](#)

Machine Learning Branca dell'intelligenza artificiale che sviluppa modelli in grado di apprendere da dati e migliorare le proprie prestazioni senza programmazione esplicita, tramite addestramento su esempi e ottimizzazione di una funzione obiettivo. [3](#)

OLAP Online Analytical Processing: modello e insieme di tecniche per analisi di dati multidimensionali, tipicamente ottimizzate per interrogazioni aggregate (slice/dice) su volumi elevati; privilegia throughput e scansioni a colonne rispetto alle operazioni transazionali OLTP. [27](#)

Overfitting Situazione in cui un modello apprende troppo dai dati di training, catturando rumore o peculiarità non generalizzabili e performando peggio su dati nuovi. [11](#)

Repository Pattern Pattern architetturale che incapsula l'accesso ai dati dietro interfacce dedicate, separando la logica di dominio dai dettagli di persistenza. [31](#)

Scrum Framework agile per la gestione di prodotti complessi, basato su iterazioni time-boxed (Sprint), ruoli definiti (Product Owner, Scrum Master, Developers) ed eventi cadenzati (Sprint Planning, Daily Scrum, Sprint Review, Sprint Retrospective) descritti dalla Scrum Guide (<https://scrumguides.org/scrum-guide.html>). [6](#)

Scrum Master Facilitatore del framework Scrum: guida il team nell'applicazione delle pratiche Scrum, rimuove impedimenti, facilita le cerimonie e promuove il miglioramento continuo. [6](#)

Similarity search Tecnica di ricerca che, dato un oggetto di query, restituisce gli oggetti più simili in base a una misura di distanza (es. cosine, L2) calcolata su rappresentazioni vettoriali come gli embedding. Usata per ricerca semantica, nearest neighbors e raccomandazione. [24](#)

Sprint Iterazione time-boxed tipica di Scrum in cui un team pianifica, realizza e rilascia un incremento di prodotto potenzialmente utilizzabile. [6](#), [8](#)

Underfitting Situazione in cui un modello è troppo semplice o poco allenato per catturare la struttura dei dati, mostrando bassa accuratezza sia su training sia su test. [11](#)

XAI Acronimo di Explainable Artificial Intelligence: disciplina che rende interpretabili i modelli di AI, fornendo motivazioni leggibili a supporto di decisioni automatiche; comprende tecniche locali e globali per evidenziare contributi delle variabili, verificare bias, soddisfare requisiti normativi e aumentare la fiducia degli utenti.. [3](#), [5](#)

Contenuto di test per generare almeno una pagina e produrre un PDF valido.

Capitolo 1

Introduzione

1.1 L'azienda

Spazio Dev S.r.l. [1](https://spaziodev.eu/) è una realtà giovane, fondata nel 2023 a Tombolo (PD), che opera come partner tecnologico per aziende industriali e commerciali con un approccio aperto e flessibile. Offre sviluppo software su misura, integrazione di sistemi e servizi cloud; progetta siti web, e-commerce e soluzioni gestionali per migliorare i processi produttivi e commerciali, con focus su integrazioni affidabili e supporto post-rilascio.



Figura 1.1: Logo di Spazio Dev

1.1.1 ErpAI

ErpAI è un progetto di Spazio Dev S.r.l. volto a realizzare un gestionale *Enterprise Resource Planning (ERP)*_G modulare, destinato a essere venduto e personalizzato per le aziende clienti. La piattaforma è pensata per coprire anagrafiche (prodotti, clienti, fornitori), flussi amministrativi (fatture di vendita e acquisto, ordini, magazzino) e componenti avanzate come schedulazione di macchine/turni e previsione delle vendite. L'architettura modulare

¹<https://spaziodev.eu/>

consente di attivare solo i domini necessari e di integrarli con sistemi esistenti tramite *Application Program Interface (API)_G*, mantenendo la flessibilità per personalizzazioni settoriali e successive estensioni di prodotto.

1.2 L'idea

Per completare l'offerta di *ErpAI*, è stata identificata la necessità di un motore di previsione vendite che supporti la pianificazione produttiva e riduca il rischio di *stock-out* o *overstock*. Il progetto nasce dal confronto con le richieste di clienti che vogliono decisioni più tempestive su produzione, approvvigionamenti e allocazione delle risorse.

Il lavoro dello stage si concentra su questa componente predittiva: partire dai dati storici delle fatture, pulirli e trasformarli in feature utili, addestrare un modello di *Machine Learning_G* per stimare la domanda futura, valutarlo usando metriche di errore e rendere i risultati disponibili al sistema *ERP_G*. Il modulo è implementato in Python ed esposto tramite FastAPI per l'integrazione via *API_G* con *ErpAI*; include spiegazioni delle previsioni con *SHapley Additive ex-Planations (SHAP)_G* per aderire ai principi di *Explainable Artificial Intelligence (XAI)_G*. La prima versione è stata sviluppata su un caso aziendale specifico e potrà essere adattata in futuro per gestire dataset e contesti di aziende differenti, variando feature, pre-processing e parametri del modello in base alle esigenze.

1.3 Organizzazione del testo

Il secondo capitolo descrive i processi aziendali e le metodologie adottate durante lo stage.

Il terzo capitolo inquadra l'azienda, il percorso di stage e gli obiettivi concordati.

Il quarto capitolo raccoglie e struttura i requisiti funzionali e non funzionali del motore di previsione.

Il quinto capitolo illustra la progettazione complessiva: architettura, pipeline dati, modello predittivo e *API_G* di servizio.

Il sesto capitolo presenta il codice sviluppato, l'organizzazione dei moduli e gli esempi principali.

Il settimo capitolo documenta la strategia di testing, i test automatici e la pipeline di integrazione continua.

L'ottavo capitolo riporta conclusioni, risultati ottenuti e possibili evoluzioni future.

Per la stesura del testo sono state adottate le seguenti convenzioni tipografiche:

- gli acronimi, le abbreviazioni e i termini ambigui o di uso non comune menzionati vengono definiti nel glossario, e questa convenzione è applicata in modo uniforme in tutto il testo;
- per la prima occorrenza dei termini riportati nel glossario viene utilizzata la forma estesa seguita dall'acronimo (ad esempio *API_G*);
- i termini in lingua straniera o facenti parti del gergo tecnico sono evidenziati con il carattere *corsivo*;
- i nomi di oggetti, funzioni, tabelle, colonne, classi e rotte *API_G* sono riportati in carattere **monospaziato**.

Capitolo 2

Processi aziendali

2.1 Organizzazione e collaborazione

Lo stage si inserisce nel progetto *ErpAI* di Spazio Dev, con l'obiettivo di realizzare il primo motore di previsione delle vendite integrato nell'*ERP_G* proprietario. Il team conta un referente tecnico aziendale, due sviluppatori e il sottoscritto; lo sviluppo del modulo di previsione e delle componenti di *XAI_G* è assegnato a me. Le procedure sono in evoluzione perché l'azienda è giovane e in crescita, ma già orientate a documentazione condivisa, verifiche automatizzate e tracciabilità delle decisioni.

2.1.1 Ambienti e strumenti di sviluppo

L'azienda consente a ciascun sviluppatore di utilizzare il proprio sistema operativo e l'*Integrated Development Environment (IDE)_G* preferito, nel rispetto delle convenzioni di progetto. La coerenza tra i team è garantita dalla containerizzazione: tutti i progetti sono eseguiti in Docker e, quando serve un ambiente di sviluppo replicabile, viene adottato DDEV. Quest'ultimo offre configurazioni preimpostate e comandi uniformi per avviare i servizi, aggiornare le dipendenze e verificare l'applicazione in locale. Ne derivano onboarding più rapido, dipendenze allineate e test eseguiti in ambienti equivalenti a quelli usati per l'integrazione continua.

2.1.2 Modello di lavoro agile

Il *framework* 2.1 adottato è Scrum, che articola il lavoro in iterazioni brevi con fasi di pianificazione, sviluppo, revisione e retrospettiva. I titolari di Spazio Dev esercitano il ruolo di Product Owner definendo priorità, obiettivi di *Sprint_G* e criteri di accettazione, e quello di *Scrum Master_G*¹ pianificando e facilitando gli eventi *Scrum_G* e rimuovendo gli impedimenti. Gli sviluppatori, incluso il sottoscritto, realizzano le attività di *Backlog_G*, rendicontano gli avanzamenti e presentano gli incrementi. Ogni *Sprint_G* settimanale si apre con la definizione del lavoro da eseguire per *ErpAI* e si chiude con la verifica degli incrementi consegnati e la retrospettiva sui miglioramenti di processo.

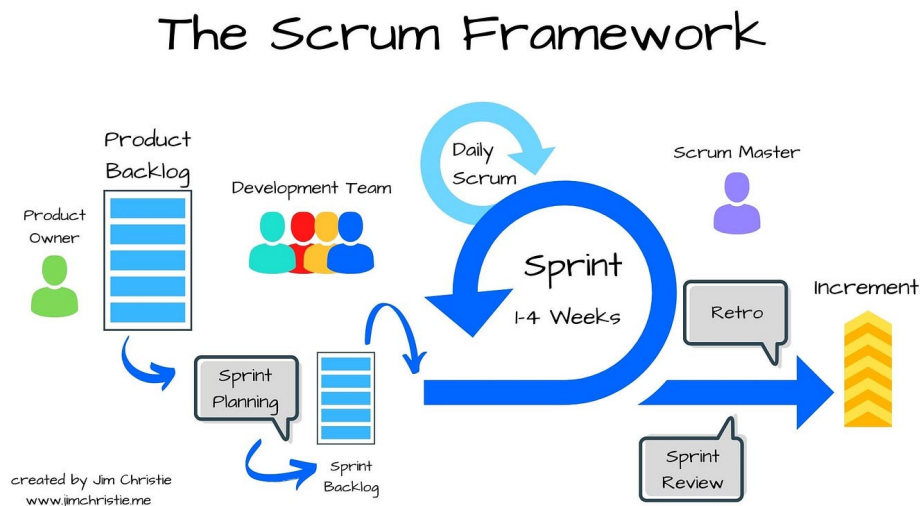


Figura 2.1: Schema semplificato del *framework* Scrum adottato

2.1.3 Comunicazione e documentazione

La comunicazione avviene su canali Telegram dedicati a ciascun progetto, dove team e referenti si scambiano aggiornamenti quotidiani, e durante le cerimonie *Scrum_G* già descritte. Per la documentazione si utilizza la sezione *Pages* di Plane, accessibile a tutti i membri del progetto, affiancata dai file nel *repository* (README, note operative). Per questo lavoro è presente anche una

¹<https://scrumguides.org/scrum-guide.html>

documentazione che descrive le *Representational State Transfer (REST)_G API_G* sviluppate.

2.2 Strumenti tecnici e controllo qualità

2.2.1 Version control e branching

Il codice è gestito con Git², sistema di controllo versione distribuito, e ospitato su Gitea³, piattaforma *self-hosted* per *repository* e *merge request*. È adottato un *Modello di branching_G* ispirato a Gitflow ma ridotto alle necessità del progetto, come illustrato in Figura 2.2:

- **main**: contiene le versioni rilasciabili e sincronizzate con l'*ERP_G*;
- **develop**: integra in modo continuo le attività in corso;
- **feature/<nome>**: per lo sviluppo di nuove funzionalità;
- **fix/<nome>**: per correggere errori individuati in test o review;
- **docs/<nome>**: per aggiornamenti alla documentazione.

Ogni branch è nominato in modo descrittivo per collegare rapidamente il lavoro all'attività tracciata.

I messaggi di commit seguono lo standard *Conventional Commits*⁴ con prefissi **feat**, **fix** e **docs**, così da facilitare la lettura della cronologia e la generazione di note di rilascio.

²<https://git-scm.com/>

³<https://about.gitea.com/>

⁴<https://www.conventionalcommits.org/en/v1.0.0/>



Figura 2.2: Schema del branching model adottato

2.2.2 Issue tracking

Le attività sono tracciate su Plane⁵ con una board *Kanban board_G* (Figura 2.3) a quattro colonne: *Backlog_G* raccoglie le attività proposte; *Todo* contiene quelle selezionate per lo *Sprint_G*; *In Progress* include il lavoro in corso; *Done* raccoglie ciò che è completato.

Ogni issue include:

- descrizione chiara dell'attività, dell'obiettivo e dell'output atteso, sia esso una nuova feature, un bugfix o un aggiornamento documentale;
- livello di priorità, con valori *Critical*, *High*, *Medium* o *Low*;
- assegnatario dell'attività, così da rendere esplicita la responsabilità e il punto di contatto;
- modulo del progetto a cui l'attività è collegata, utile per aggregare il lavoro per componente e verificare l'impatto.

⁵<https://plane.so>

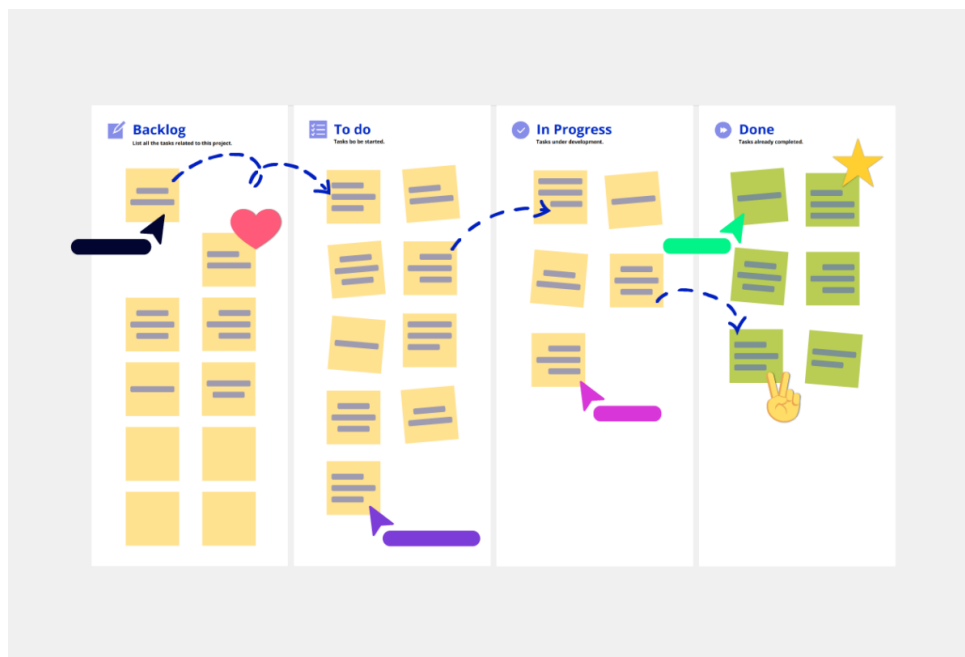


Figura 2.3: Schema semplificato della board Kanban su Plane

Capitolo 3

Descrizione dello stage

3.1 Introduzione al progetto

Il modulo di previsione sviluppato durante lo stage stima la domanda futura di prodotti finiti per supportare la pianificazione *Material Requirements Planning (MRP)_G* di *ErpAI*. Le stime si basano sullo storico delle fatture di vendita e vengono propagate lungo le distinte base per evidenziare i fabbisogni di semi-lavorati e materie prime. I risultati vengono messi a disposizione dei planner come proposte rivedibili, affiancando inizialmente il processo manuale.

Obiettivi principali:

- ridurre *stock-out* e fermate impianto anticipando materiali critici;
- limitare *overstock* e costi di magazzino evitando produzioni o acquisti non necessari;
- fornire previsioni integrabili con il modulo di *planning* e l'interfaccia utente per approvare o modificare le proposte.

3.2 Analisi preventiva dei rischi

Durante la fase iniziale sono stati individuati alcuni rischi specifici per il modulo di previsione e la sua integrazione:

1. Qualità e quantità dei dati storici

Descrizione: dati di vendita incompleti o disomogenei possono ridurre l'accu-

ratezza del modello.

Soluzione: pulizia e normalizzazione dei dataset, regole di validazione condivise con i referenti di business.

2. Integrazione con l'*ERP_G* e le *API_G*

Descrizione: ritardi o incongruenze nello scambio dati tra modulo di previsione, *planning* e *ERP_G*.

Soluzione: contratto *API_G* documentato (OpenAPI), ambiente di staging per test end-to-end prima del rilascio.

3. Performance e tempi di risposta

Descrizione: tempi di addestramento o interrogazione troppo lunghi per l'uso operativo.

Soluzione: limitazione delle feature non essenziali, caching dei risultati, pianificazione di job batch fuori orario.

4. Qualità di generalizzazione del modello

Descrizione: rischio di *Overfitting_G* o *Underfitting_G* a causa di dati sporchi o parametrizzazione non ottimale.

Soluzione: pulizia e normalizzazione dei dati, regolarizzazione e ottimizzazione degli iperparametri per bilanciare complessità e capacità predittiva.

5. Adozione da parte dei planner

Descrizione: diffidenza verso i risultati del modello e ritardi nell'uso in produzione.

Soluzione: esecuzione parallela al processo manuale per più cicli, raccolta *feedback* e correzioni iterative.

3.3 Pianificazione

3.3.1 Modalità di stage e interazione

Lo stage si svolge in presenza, con allineamenti quotidiani e tre *touchpoint* settimanali con il tutor aziendale per supporto tecnico e validazione degli step.

La comunicazione avviene di persona; il tutor fornisce revisione del codice, risoluzione problemi e verifica delle scelte progettuali.

3.3.2 Pianificazione settimanale

La Tabella 3.1 riporta la pianificazione delle attività settimanali concordata con il tutor aziendale all'inizio dello stage.

Settimana	Obiettivi/Attività	Ore
1	Analisi requisiti e rischi; studio delle tecnologie da utilizzare.	40
2	Progettazione e revisione della documentazione esistente.	40
3	Codifica del progetto.	40
4	Codifica del progetto.	40
5	Codifica del progetto.	40
6	Codifica del progetto.	40
7	Codifica dei test e stesura della documentazione.	40
8	Preparazione della presentazione finale, test conclusivi e chiusura attività.	20

Tabella 3.1: Pianificazione settimanale delle attività di stage.

Capitolo 4

Analisi dei requisiti

4.1 Casi d'uso

In avvio di stage sono stati definiti requisiti e interazioni minime tra chi usa il gestionale e il motore di previsione, così da focalizzare solo i casi d'uso necessari alle previsioni.

4.1.1 Attori individuati

Gli attori che interagiscono con il motore di previsione sono:

- **Utente finale:** usa l'interfaccia di *ErpAI* per avviare previsioni e consultare i risultati.
- **Gestionale *ErpAI*:** applicazione principale che inoltra le richieste al modulo di previsione via *API_G* protette, gestisce autenticazione e memorizza/mostra i risultati.
- **Modulo di previsione:** modulo che aggiorna i dati, allena il modello, genera previsioni e notifica *ErpAI* via *Callback / Webhook_G*.
- **Modulo chatbot:** modulo che può interrogare il modulo di previsione via *API_G* per ottenere previsioni e spiegazioni da usare nelle risposte.

L'interazione è orchestrata da *ErpAI*, che media verso il modulo di previsione e il chatbot.

4.1.2 Casi d'uso

La Tabella 4.1 riporta i casi d'uso principali individuati per il modulo di previsione.

Identificativo	Descrizione
UC1	Attori: Utente finale.
	Scopo: Avviare una richiesta di previsione.
	Pre-condizione: Autenticazione effettuata su <i>ErpAI</i> .
	Post-condizione: Richiesta creata e inoltrata a <i>ErpAI</i> .
UC1.1	Attori: Utente finale.
	Scopo: Selezionare l'intervallo temporale per la previsione.
	Pre-condizione: Interfaccia di <i>ErpAI</i> aperta sulla schermata di previsione.
	Post-condizione: Intervallo impostato.
UC1.2	Attori: Utente finale.
	Scopo: Confermare e inviare la richiesta.
	Pre-condizione: Parametri configurati (intervallo selezionato).
	Post-condizione: Richiesta inviata a <i>ErpAI</i> .
UC2	Attori: Gestionale <i>ErpAI</i> .
	Scopo: Inoltrare la richiesta al modulo di previsione.
	Pre-condizione: Richiesta ricevuta dal frontend, <i>API_G</i> del modulo raggiungibile.
	Post-condizione: Richiesta inoltrata.
UC2.1	Attori: Gestionale <i>ErpAI</i> .
Continua...	

Continua...

Identificativo	Descrizione
	Scopo: Richiedere token al modulo di previsione.
	Pre-condizione: Credenziali client configurate.
	Post-condizione: Token ricevuto oppure errore di autenticazione.
UC2.2	Attori: Gestionale <i>ErpAI</i> .
	Scopo: Inviare la richiesta con token al modulo di previsione.
	Pre-condizione: Token valido, parametri di previsione disponibili.
	Post-condizione: Richiesta accettata o errore ricevuto.
UC3	Attori: Modulo di previsione.
	Scopo: Processare la previsione e notificare <i>ErpAI</i> .
	Pre-condizione: Richiesta UC2.2 ricevuta e valida.
	Post-condizione: Previsione elaborata o errore rilevato.
UC3.1	Attori: Modulo di previsione.
	Scopo: Elaborare dati e generare previsione.
	Pre-condizione: Richiesta valida.
	Post-condizione: Previsione pronta oppure errore registrato.
UC3.2	Attori: Modulo di previsione.
	Scopo: Notificare il completamento a <i>ErpAI</i> .
	Pre-condizione: Processo terminato (successo o fallimento).

Continua...

Continua...

Identificativo	Descrizione
	Post-condizione: Stato aggiornato in <i>ErpAI</i> .
UC3.E	Attori: Modulo di previsione.
	Scopo: Notificare errore a <i>ErpAI</i> .
	Pre-condizione: Errore registrato durante UC3.1.
	Post-condizione: Stato marcato come fallito in <i>ErpAI</i> .
UC4	Attori: Modulo Chatbot.
	Scopo: Richiedere contesto previsione.
	Pre-condizione: Chatbot autorizzato verso le <i>API_G</i> del modulo di previsione.
	Post-condizione: Richieste pronte o dati ricevuti/errore gestito.
UC4.1	Attori: Modulo Chatbot.
	Scopo: Preparare richiesta e token.
	Pre-condizione: Credenziali client configurate.
	Post-condizione: Token ricevuto oppure errore di autenticazione.
UC4.2	Attori: Modulo Chatbot.
	Scopo: Recuperare dati previsionali via <i>API_G</i> .
	Pre-condizione: Token valido, ID previsione noto.
	Post-condizione: Dati ricevuti oppure errore gestito.
UC4.E	Attori: Modulo Chatbot.
	Scopo: Gestire errore <i>API_G</i> .

Continua...

Continua...

Identificativo	Descrizione
	Pre-condizione: Errore nelle chiamate di UC4.1 o UC4.2.
	Post-condizione: Errore registrato, risposta di errore al chiamante.

Tabella 4.1: Tabella dei casi d'uso

4.1.3 Diagrammi dei casi d'uso

I diagrammi seguenti descrivono i flussi dei casi d'uso presentati (Figure 4.1, 4.2, 4.3 e 4.4).

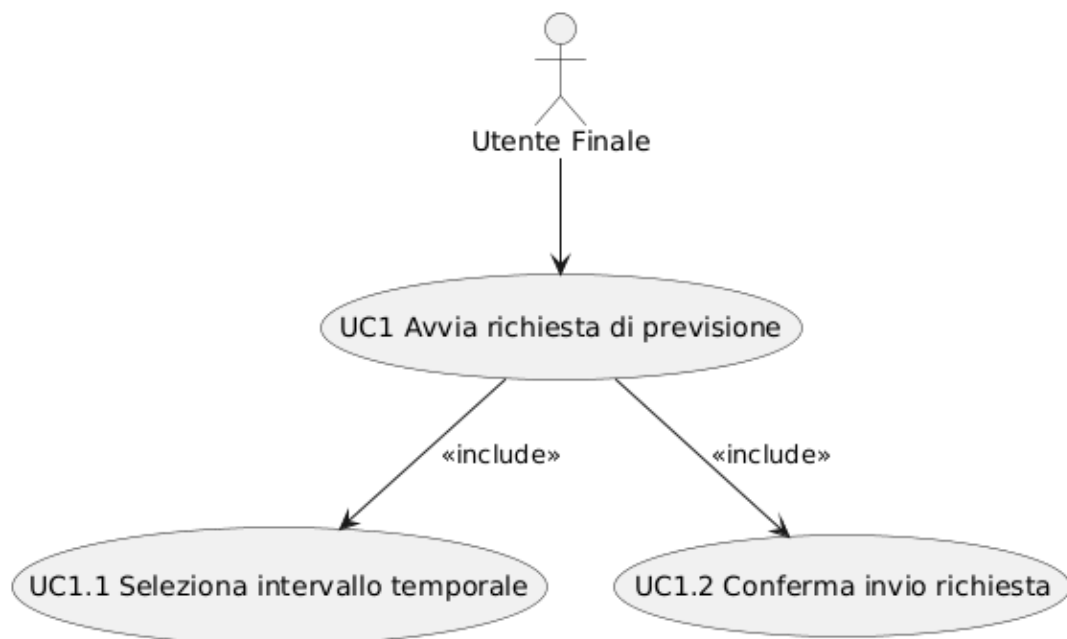


Figura 4.1: UC.1: avvio e invio richiesta di previsione

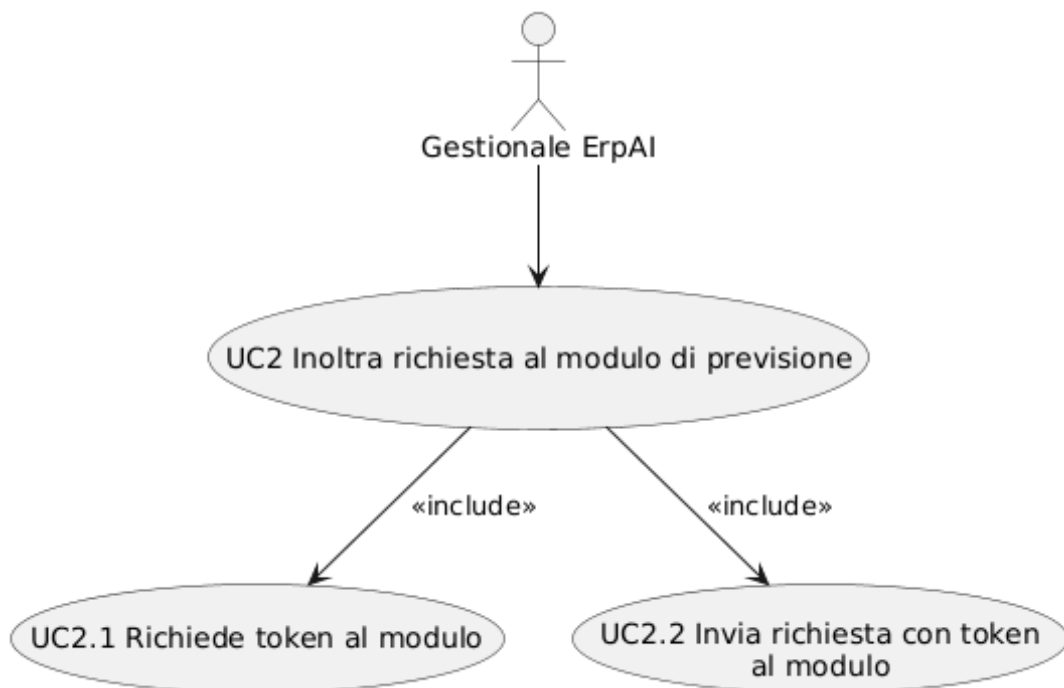


Figura 4.2: UC.2: inoltro della richiesta da *ErpAI* al modulo di previsione

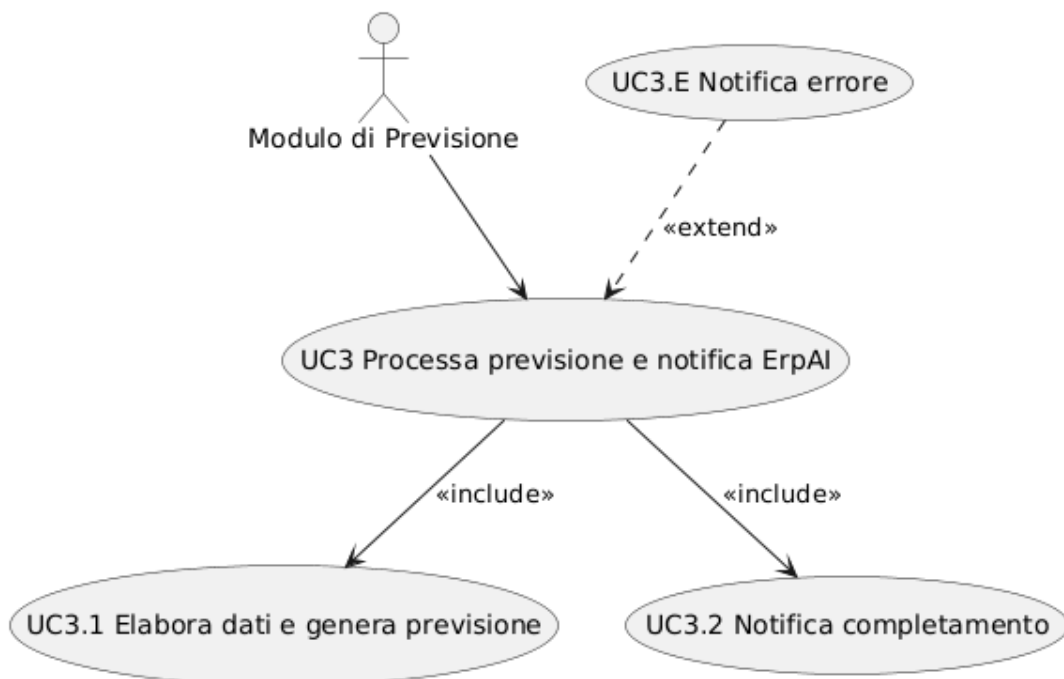


Figura 4.3: UC.3: elaborazione e notifica da parte del modulo di previsione

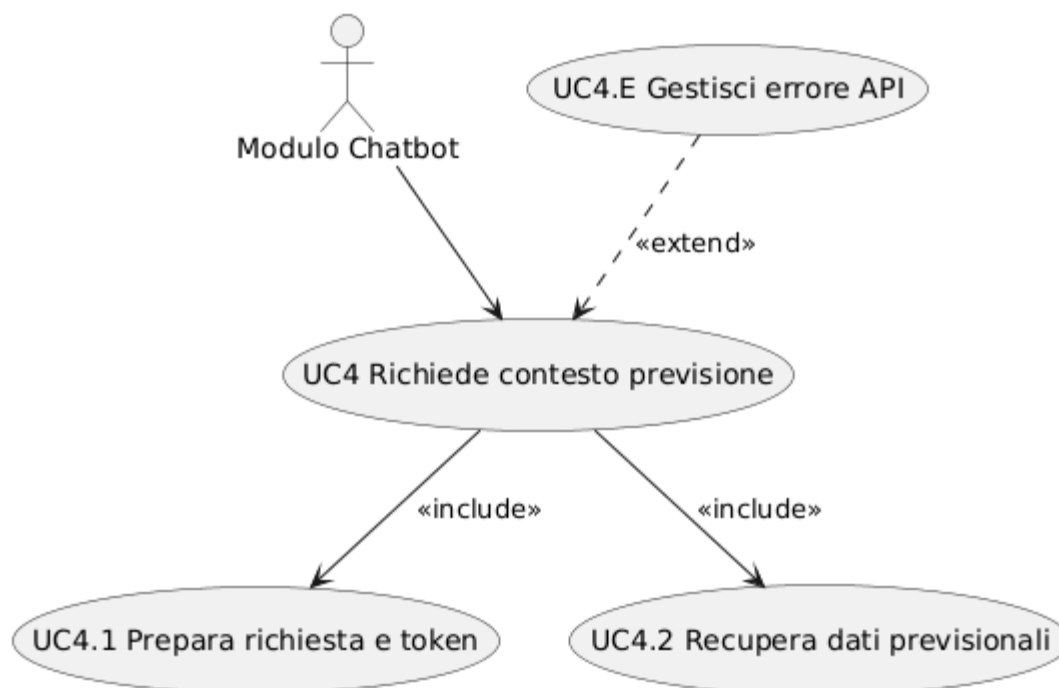


Figura 4.4: UC.4: interazione del modulo chatbot con il modulo di previsione

4.2 Requisiti

Questa sezione sintetizza i requisiti del modulo di previsione. Gli item di *Backlog_G* sono stati raccolti e prioritizzati insieme al Product Owner, etichettati per criticità e condivisi come base di accordo con gli stakeholder.

4.2.1 Codifica requisiti

Ogni requisito del modulo di previsione per *ErpAI* è codificato nel seguente formato:

- **Formato:** R.x.y
- **Tipologia (x):** O = obbligatorio; D = desiderabile; F = facoltativo
- **Progressivo (y):** valore numerico progressivo a partire da 0

4.2.2 Elenco requisiti

La tabella 4.2.2 riporta l'elenco completo dei requisiti raccolti per il modulo di previsione.

ID	Descrizione
R.O.0	Il sistema deve stimare quante vendite (in pezzi) l'azienda effettuerà nei periodi futuri configurabili, partendo dai dati <i>ERP_G</i> .
R.O.1	Le esecuzioni di previsione devono essere gestite come job asincroni con stati tracciati (creazione, avanzamento, successo/fallimento) e consultabili.
R.O.2	Le previsioni devono essere salvate in archivi aziendali con identificativo di run, righe previsionali e controlli su duplicati/chiavi univoche.
R.O.3	I dati <i>ERP_G</i> devono essere sincronizzati periodicamente in un data store locale, con normalizzazione e indici per l'uso del modello.
R.O.4	L'accesso alle funzionalità di previsione e alle consultazioni deve essere protetto tramite autenticazione e validazione di token/ <i>JSON Web Key Set (JWKS)_G</i> .
R.O.5	Il sistema deve notificare un endpoint applicativo configurabile al termine dei job (successo o errore).
R.O.6	La gestione degli errori non deve interrompere l'elaborazione: i task in background devono gestire eccezioni senza bloccare il servizio.
R.O.7	Le <i>API_G</i> devono esporre documentazione coerente e accessibile per l'integrazione con i servizi aziendali.
R.O.8	Il servizio deve essere distribuibile in modo riproducibile (container/docker-compose) con configurazione via variabili ambiente.
R.D.0	Il sistema dovrebbe produrre spiegazioni delle previsioni (es. feature importanti) e associarle ai risultati salvati.
R.D.1	Il sistema dovrebbe offrire un contesto consultabile (ricerca/testo) sulle previsioni per supportare chatbot o analisi esplorativa.

Continua...

Continua...

ID	Descrizione
R.D.2	Il modello dovrebbe calcolare e tracciare metriche di accuratezza per ogni run, con target di errore medio sotto il 20%.
R.D.3	Il sistema dovrebbe supportare analisi “what-if”, consentendo di variare parametri e confrontare scenari di previsione.
R.D.4	Il sistema dovrebbe supportare logica di validazione automatica (test su dati di holdout) prima di promuovere un nuovo modello in produzione.
R.F.0	Potrebbe essere utile fornire esportazioni in formati aggiuntivi (<i>Comma-Separated Values (CSV)</i> _G /Excel/ <i>JavaScript Object Notation (JSON)</i> _G) per integrazioni non <i>API</i> _G .
R.F.1	Potrebbe essere utile offrire notifiche opzionali via email/Slack oltre al <i>Callback / Webhook</i> _G configurabile.
R.F.2	Potrebbe essere utile prevedere dashboard sintetiche per monitorare trend e performance del modello.

Tabella 4.2: Elenco dei requisiti

Capitolo 5

Progettazione

Una parte consistente del monte ore iniziale dello stage è stata investita nella fase di progettazione: definizione dell’architettura del modulo di previsione, individuazione dei confini funzionali e selezione degli strumenti. Le prime settimane sono state dedicate alla prova comparativa delle tecnologie (modello, *API_G*, orchestrazione), allo studio della documentazione ufficiale e alla costruzione di prototipi rapidi; ciò ha permesso di arrivare alla fase successiva con scelte già validate e di avviare lo sviluppo su basi stabili.

5.1 Tecnologie e strumenti

In questa sezione sono elencate le principali tecnologie e librerie adottate per lo sviluppo del modulo di previsione, con una breve descrizione delle loro funzionalità.

5.1.1 Linguaggi di programmazione

- **Python**¹: linguaggio di programmazione interpretato, ad alto livello e tipizzato dinamicamente, progettato per leggibilità e rapidità di sviluppo. Offre un vasto ecosistema di librerie per calcolo scientifico, machine learning e sviluppo applicativo.

¹<https://www.python.org/>

- **SQL**: è un linguaggio dichiarativo standardizzato per definire, interrogare e manipolare dati all'interno di database relazionali. Fornisce strumenti per creare schemi, gestire transazioni ed eseguire query complesse in modo ottimizzato. È il mezzo principale per accedere e gestire dati strutturati in sistemi informativi.

5.1.2 Framework e server web

- **FastAPI²**: FastAPI è un *framework* web Python ad alte prestazioni per la creazione di *API*_G basate su standard come OpenAPI e JSON Schema. Utilizza tipizzazione Python e Pydantic per validazione automatica e autogenerazione della documentazione.
- **Uvicorn**: server *ASGI*_G leggero e ad alte prestazioni, basato su uvloop e httptools, progettato per applicazioni web asincrone in Python. Fornisce un runtime veloce e a bassa latenza ideale per *framework* come FastAPI.

5.1.3 Database e ORM

- **SQLAlchemy**: toolkit SQL e ORM per Python che fornisce un livello di astrazione avanzato per interagire con database relazionali.
- **Alembic³**: strumento di database migration per SQLAlchemy che gestisce in modo versionato l'evoluzione degli schemi relazionali. Permette di generare, applicare e monitorare migrazioni tramite script incrementali.
- **DuckDB**: è un database analitico in-process orientato alle query OLAP, progettato per esecuzioni ad alte prestazioni direttamente all'interno dell'applicazione. Utilizza un motore vettorizzato e columnar per elaborare rapidamente grandi dataset senza necessità di un server esterno.
- **ChromaDB**: database vettoriale progettato per l'indicizzazione e la ricerca semantica tramite *Embedding*_G ad alta dimensionalità. Fornisce

²<https://fastapi.tiangolo.com/>

³<https://alembic.sqlalchemy.org/>

storage ottimizzato, funzioni di *similarity search*_G e gestione di metadata per applicazioni *LLM*_G e retrieval-augmented.

5.1.4 Autenticazione e sicurezza

- **PyJWT**⁴: libreria Python per creare, firmare e validare *JSON Web Token* (*JWT*)_G, utilizzati per autenticazione e scambio sicuro di informazioni.

5.1.5 Machine Learning e Data Science

- **Pydantic**⁵: libreria Python per la validazione e la gestione di dati basata su annotazioni di tipo. Consente di definire modelli di dati con tipi forti, validazione automatica e serializzazione/deserializzazione in modo semplice ed efficiente.
- **Pandas**: libreria Python per la manipolazione e l'analisi di dati strutturati, basata su oggetti fondamentali come DataFrame e Series. Offre strumenti ad alte prestazioni per filtrare, trasformare, aggregare e pulire dataset.
- **NumPy**: libreria Python per il calcolo scientifico che fornisce array n-dimensionali ad alte prestazioni e operazioni vettorializzate. Include funzioni per algebra lineare, statistica e trasformazioni numeriche ottimizzate in C.
- **scikit-learn**: libreria Python per il machine learning che offre algoritmi efficienti per classificazione, regressione, clustering e riduzione della dimensionalità. Fornisce *API*_G uniformi, strumenti per preprocessing, valutazione dei modelli e pipeline.
- **XGBoost**⁶: libreria Python ottimizzata per gradient boosting che implementa alberi decisionali ad alte prestazioni, pensata per competizioni

⁴<https://pyjwt.readthedocs.io/>

⁵<https://pydantic.dev/>

⁶<https://xgboost.ai/>

di machine learning e applicazioni industriali. Utilizza parallelismo, pruning avanzato e ottimizzazioni su memoria e *GPU_G*. È nota per l'elevata accuratezza e l'efficienza su grandi dataset tabulari.

- *SHAP_G*⁷: libreria Python che fornisce metodi basati sui valori di Shapley per interpretare modelli di machine learning. Calcola contributi locali e globali delle feature alle predizioni. È ampiamente utilizzata per garantire trasparenza, explainability e analisi di modelli complessi come XGBoost

5.1.6 Strumenti di testing e deployment

- **Pytest**: *framework* di testing per Python che permette di scrivere test semplici, modulari e scalabili tramite un approccio basato su funzioni e fixture. Supporta parametric testing, plugin estendibili e rilevamento automatico dei test.
- **Docker**⁸: piattaforma di containerizzazione che consente di impacchettare applicazioni e dipendenze in ambienti isolati e portabili chiamati container. Utilizza un motore leggero basato su immagini stratificate per garantire coerenza tra ambienti di sviluppo, test e produzione. Facilita il deployment scalabile e riproducibile di applicazioni moderne.

5.2 Architettura

L'architettura complessiva del sistema 5.1 è stata progettata secondo principi di modularità e componibilità, in modo da permettere l'evoluzione indipendente dei singoli blocchi funzionali. Il modulo *ErpAI-Forecasting* si inserisce come componente aggiuntivo della piattaforma *ErpAI* e fornisce funzionalità di previsione basate su modelli di machine learning. Tutte le comunicazioni verso il modulo avvengono tramite l'*API_G* Gateway, che gestisce la validazione dei to-

⁷<https://shap.readthedocs.io/>

⁸<https://www.docker.com/>

ken JWT_G presso l'Auth Server, l'instradamento delle richieste e l'esposizione controllata delle API_G pubbliche e interne.

L'orchestrazione vera e propria è realizzata all'interno del Modulo di Previsione, che combina tre elementi chiave: dati ERP_G sincronizzati tramite DuckDB, un modello XGBoost ottimizzato per il dominio applicativo, e un sistema di persistenza distribuito che comprende Prevision DB e Chroma DB. Questa architettura permette di separare nettamente l'elaborazione analitica (DuckDB), l'inferenza e l'explainability (Modulo di Previsione) e la persistenza (Prevision DB / Chroma DB). La presenza di contratti ben definiti (schema delle API_G $REST_G$, modelli Pydantic, formati dei dataset e versioning dei modelli) garantisce la possibilità di sostituire o aggiornare ciascun componente senza introdurre impatti a catena sugli altri servizi.

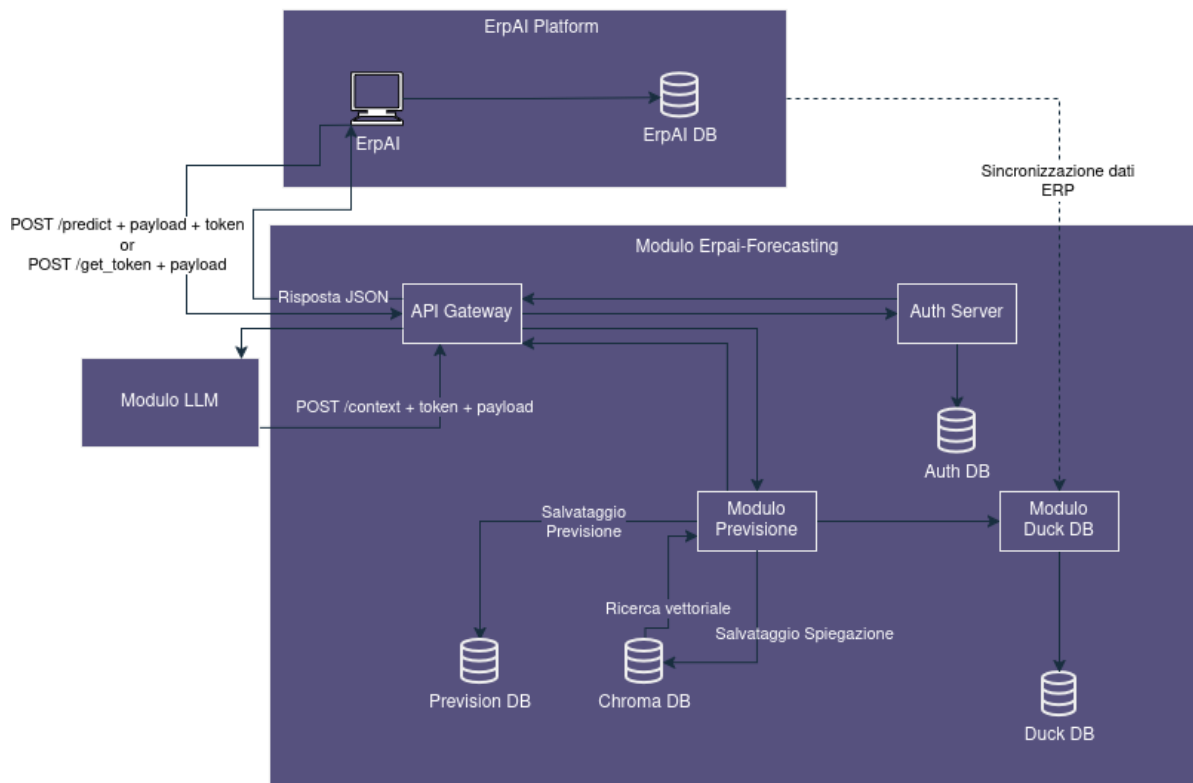


Figura 5.1: Architettura complessiva del modulo ErpAI-Forecasting

5.2.1 API Gateway

Il sistema utilizza Kong⁹ come *API_G* Gateway per garantire un unico punto di ingresso alle *API_G* esposte dal modulo *ErpAI-Forecasting*. Kong gestisce instradamento, rate limiting, logging e policy di sicurezza, consentendo di centralizzare funzioni trasversali che altrimenti dovrebbero essere replicate nei singoli servizi. Ogni richiesta proveniente da *ErpAI* o dal Modulo *LLM_G* viene validata e inoltrata al Modulo di Previsione solo dopo la verifica del token *JWT_G*.

5.2.2 Auth Server

L'Auth Server è responsabile dell'autenticazione e dell'emissione dei token *JWT_G*. Espone endpoint per la richiesta di credenziali, la validazione dei token e la gestione di client applicativi autorizzati. Si appoggia ad Auth DB per memorizzare utenti, ruoli e chiavi pubbliche/privati necessarie alla firma e alla verifica dei token. L'*API_G* Gateway consulta l'Auth Server (o i suoi metadati *JWKS_G*) per validare l'autenticità delle richieste in ingresso.

5.2.3 Modulo DuckDB

DuckDB è il database embedded, analitico e in-process orientato a query *OLAP_G* che alimenta il forecasting e ospita la replica dei dati *ERP_G*. La configurazione legge l'URL da `DUCKDB_URL` (o `DUCKDB_DATABASE_URL` nel servizio di sync), costruisce il percorso su filesystem e, se la directory non è scrivibile, ripiega su un file locale in sola lettura quando già esistente. Il *repository* di forecasting espone `fetch_fact_invoices`, che restituisce `fact_invoices` come DataFrame Pandas da usare per normalizzazione e preparazione dei dataset.

- **Sincronizzazione:** un servizio di orchestrazione preleva i dati *ERP_G* attraverso i seguenti metodi:
 - **fetch_invoices:** sincronizza la tabella `invoices` con le informazioni delle fatture di vendita emesse.

⁹<https://konghq.com/>

- **fetch_articles**: sincronizza la tabella `articles` con tutti gli articoli in vendita.
 - **fetch_customers**: sincronizza la tabella `customers` con l'anagrafica clienti.
 - **fetch_vat_options**: sincronizza la tabella `vat_options` con le aliquote IVA.
 - **fetch_article_invoice**: sincronizza la tabella `article_invoice` con le righe degli articoli presenti nelle fatture.
- **Costruzione di `fact_invoices`**: dopo la sincronizzazione viene ricreata la tabella `fact_invoices`, che aggrega le informazioni rilevanti per il forecasting calcolando metriche come il totale fatturato per cliente, articolo e periodo.

5.2.4 Modulo di Previsione

Il Modulo di Previsione orchestrato da FastAPI gestisce l'intero ciclo di inferenza, dall'accesso ai dati fino alla persistenza dei risultati. Gli endpoint *REST_G* validano il payload (Pydantic), avviano un job tracciato e coordinano i servizi sottostanti:

- **Acquisizione dati**: lettura delle `fact_invoices` da DuckDB, normalizzazione di colonne/tempi e copertura mensile per articolo; filtri opzionali sulla data di cutoff.
- *Feature engineering_G*: costruzione del dataframe utilizzato per il training del modello e aggiunta di feature derivate come lag, rolling, trend stagionali e indicatori temporali.
- **Inferenza XGBoost**: allenamento del modello XGBoost sui dati raccolti e generazione delle previsioni per il periodo richiesto con calcolo delle metriche di accuratezza come *Mean Absolute Percentage Error (MAPE)_G*, *Weighted Absolute Percentage Error (WAPE)_G* e *Root Mean Squared Error (RMSE)_G*.

- **Spiegazioni $SHAP_G$:** calcolo dei valori $SHAP_G$ per ogni previsione generata, con estrazione delle feature più influenti.
- **Persistenza:** i risultati numerici, le spiegazioni e le metriche vengono salvati in Prevision DB e Chroma DB per consentire consultazioni future e audit dei processi.
- **Notifiche e callback:** al termine del job di previsione viene inviata una notifica di completamento ad *ErpAI* tramite *Callback / Webhook $_G$* *Hypertext Transfer Protocol (HTTP) $_G$* , includendo lo stato e i riferimenti ai risultati salvati.

5.2.5 Prevision DB

Prevision DB conserva job, run di previsione e righe di output. Le principali tabelle sono:

- **jobs:** contiene i job di previsione ed è composta da:
 - **id:** identificativo univoco del job.
 - **status:** stato del job (in coda, in esecuzione, completato, fallito).
 - **detail:** dettaglio/errore se presente.
 - **dataset__id:** riferimento al dataset sincronizzato usato dal job.
 - **timestamp:** creazione, avvio, fine ed ultimo aggiornamento.
- **forecast__runs:** registra le esecuzioni di previsione, con:
 - **id:** identificativo del run.
 - **mape:** metrica di errore della previsione usata per valutare la run.
 - **created__at:** data e ora di creazione della run.
- **predictions__xgboost:** contiene le righe previsionali, con:
 - **forecast__id:** riferimento alla run che ha generato la previsione.
 - **article__id** e **article__name:** articolo a cui si riferisce la previsione.

- **prediction__date**: data della previsione.
- **predicted__quantity**: quantità stimata.

La struttura dei dati consente di collegare job, run e singole previsioni, supportando consultazioni e analisi successive.

5.2.6 Chroma DB

Chroma DB è l'archivio vettoriale che memorizza *Embedding_G*, previsioni e spiegazioni, organizzate in due collection collegate da un ID di run nei metadati:

- **prevision__collection**: contiene le previsioni (article_id, article_name, data, quantità prevista) con *Embedding_G* e metadati della run.
- **explanation__collection**: memorizza le spiegazioni *SHAP_G* per ogni previsione, con metadati che riferiscono alla stessa run e all'articolo.

Le due collection permettono ricerche semantiche su risultati e spiegazioni e il collegamento rapido alla run e all'articolo, a supporto del Modulo *LLM_G* tramite *Embedding_G*.

5.2.7 Auth DB

Auth DB gestisce le credenziali dei client applicativi tramite la tabella **clients**, composta da:

- **id**: identificativo della riga.
- **client__id**: identificativo pubblico del client.
- **client__secret**: segreto usato per autenticare il client.
- **is__active**: flag che abilita o disabilita il client.

L'Auth Server consulta questa tabella per verificare e autorizzare i client in ingresso.

5.3 Design pattern adottati

In questa sezione sono riassunti i pattern architetturali e di inversione del controllo adottati, con l'obiettivo di separare le responsabilità, semplificare lo sviluppo e rendere il modulo facilmente testabile e manutenibile.

5.3.1 Architettura a layer

L'architettura del modulo di previsione è organizzata a layer, ogni layer ha responsabilità ben definite e comunica solo con i layer adiacenti in un approccio *closed layer*. Il vantaggio principale di questa struttura è la separazione delle responsabilità, che consente di isolare la logica di business dalla gestione delle *API_G* e dall'accesso ai dati. **Layer implementati:**

- **API Layer:** espone *API_G REST_G* (FastAPI), valida i payload e avvia il flusso; nessuna logica di dominio né accesso diretto ai dati.
- **Business Layer:** logica business del modulo di previsione, riceve *DTO_G* dalle *API_G* e effettua le operazioni necessarie orchestrando i servizi sottostanti.
- **Persistence Layer:** collega il business layer ai database esterni tramite *repository* dedicati; incapsula logica di accesso ai dati e query SQL.
- **Database Layer:** contiene i database del sistema. dove i dati sono effettivamente memorizzati.

5.3.2 Repository pattern

Il sistema adotta il *Repository Pattern_G* per isolare completamente la logica di accesso ai dati dai servizi applicativi. Ogni database o storage esterno (relazionale, vettoriale o analitico) è incapsulato in un *repository* dedicato che espone metodi di alto livello per *CRUD_G* e query di dominio. Questa astrazione mantiene i servizi indipendenti dall'implementazione fisica del dato, semplifica

l'integrazione con backend diversi, migliora la testabilità tramite mock e accelera la sostituzione o l'evoluzione dei motori di persistenza. Lo snippet 5.1 mostra un esempio di *repository* per Chroma DB.

- **ForecastRepository**: accesso a Prevision DB (creazione job, salvataggio run, metriche, righe previsionali).
- **DuckDBRepository**: accesso a DuckDB (`fact_invoices` come Data-Frame Pandas).
- **ChromaRepository**: accesso a Chroma DB (salvataggio/interrogazione embedding e metadati semantici).

```
1 from chromadb.api import ClientAPI
2
3 class ChromaRepository:
4     def __init__(self, client: ClientAPI):
5         self._client = client
6
7     def upsert_documents(self, collection: str, ids: list[str], docs:
8         ↪ list[str]) -> None:
9         coll = self._client.get_or_create_collection(name=collection)
10        coll.upsert(ids=ids, documents=docs)
11
12    def search(self, collection: str, text: str, n_results: int = 5):
13        coll = self._client.get_collection(name=collection)
14        if coll is None:
15            return None
16        return coll.query(query_texts=[text], n_results=n_results,
17            ↪ include=["documents", "metadatas"])
```

Codice 5.1: Esempio di *repository* per Chroma.

5.3.3 Inversione del controllo

Il modulo di previsione utilizza il principio di Inversion of Control tramite Dependency Injection per gestire le relazioni tra i vari componenti del sistema. In pratica, i servizi non creano direttamente gli oggetti di cui hanno bisogno, ma li ricevono già pronti dall'esterno, ad esempio tramite il costruttore. Questo approccio rende il codice più ordinato e facile da mantenere, perché ogni componente è meno dipendente dagli altri. Permette inoltre di sostituire rapidamente una parte del sistema senza modificare il resto dell'applicazione, e semplifica molto anche la fase di test, dove è possibile fornire versioni simulate delle dipendenze. Lo snippet 5.2 mostra un esempio di dependency injection in `ForecastService`.

```
1 class ForecastingService:
2     def __init__(
3         self,
4         *,
5         xgb: XGBoostService | None = None,
6         shap: ShapService | None = None,
7         chroma_repo: ChromaRepository | None = None,
8         db_repo: DBRepository | None = None,
9         normalize: DataNormalizeService | None = None,
10    ):
11        self._xgb = xgb or XGBoostService()
12        self._shap = shap or ShapService()
13        self._chroma = chroma_repo or ChromaRepository()
14        self._db = db_repo or DBRepository()
15        self._norm = normalize or DataNormalizeService()
16
17        ...
```

Codice 5.2: Esempio di dependency injection.

Capitolo 6

Codifica

Gran parte del periodo di stage è stata dedicata allo sviluppo del codice che realizza il sistema di previsione descritto nei capitoli precedenti. In questo capitolo vengono presentate l'organizzazione della codebase e le principali logiche applicative che la compongono.

6.1 Struttura del codice

La struttura delle directory del progetto è organizzata per separare chiaramente i vari componenti e facilitare la navigazione del codice [6.1](#). Di seguito è riportata una descrizione delle principali cartelle e file presenti nel *repository*.

6.1.1 Root

- `ErpAI-Forecasting/`: cartella principale del progetto contenente tutto il codice sorgente.
 - `.gitea/`: cartella contenente le configurazioni per l'integrazione continua con Gitea.
 - `auth-server/`: modulo di autenticazione.
 - `forecasting/`: modulo principale del sistema.
 - `duckdb/`: servizio di sincronizzazione dati *ERP_G* con scheduler mensile.

- `kong/`: cartella contenente la configurazione del gateway *API_G* che espone i servizi interni.
- `openapi/`: servizio che pubblica la documentazione *API_G*.
- `.env`: variabili d'ambiente.
- `.gitignore`: file di esclusione git.
- `alembic.ini`: configurazione di Alembic per le migrazioni del database.
- `compose.base.yml`: definizione Docker Compose base.
- `compose.yml`: definizione Docker Compose principale.
- `Dockerfile`: build dell'immagine principale.
- `pytest.ini`: configurazioni per i test.
- `README.md`: documentazione di progetto.

6.1.2 forecasting

Approfondiamo il modulo di previsione in quanto cuore del sistema e oggetto principale di questo lavoro di tesi.

- `app/`: cartella principale del modulo di previsione.
 - `api/`: cartella che contiene i router FastAPI per gli endpoint *REST_G*.
 - `db/`: cartella che contiene engine e setup DB locale.
 - `dependencies/`: cartella che contiene la configurazione per la dependency injection.
 - `models/`: cartella che contiene i modelli Pydantic e SQLAlchemy per payload e tabelle.
 - `repositories/`: cartella che contiene le implementazioni del repository pattern per l'accesso ai database esterni.
 - `services/`: cartella che contiene le classi che implementano la logica di business e orchestrazione del flusso di previsione.

- `chroma-data/`: cartella che contiene i dati persistenti di Chroma DB.
- `graphs/`: cartella che contiene gli output grafici *SHAP_G* generati.
- `migrations/`: cartella che contiene gli script Alembic per la gestione delle migrazioni del database.
- `tests/`: cartella che contiene la suite di test.
- `compose.forecasting.yml`: definizione Docker Compose specifica per il servizio forecasting.
- `Dockerfile`: build dell'immagine del servizio forecasting.
- `requirements.txt`: dipendenze Python.

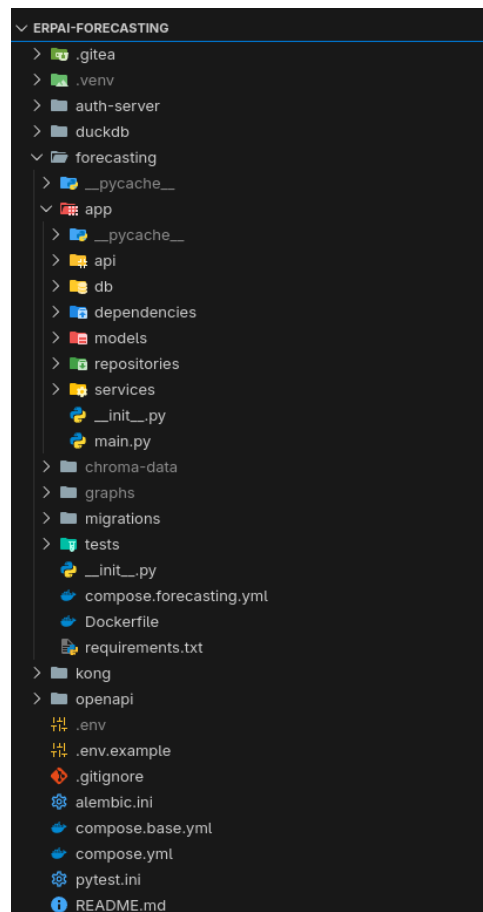


Figura 6.1: Struttura delle directory del progetto.

6.2 Modelli dei dati

Nella cartella `models/` si trovano i modelli utilizzati per descrivere in modo tipizzato i dati che attraversano l'applicazione: gli schema Pydantic definiscono i payload di richiesta e risposta esposti dalle [API](#) FastAPI, mentre i modelli `SQLModel` rappresentano le tabelle e le relazioni persistite nei database operazionali. Di seguito sono riportati i principali modelli implementati, con una breve descrizione dei campi e il codice di riferimento.

6.2.1 Modelli Pydantic

6.2.1.1 Article

Questo modello definisce e valida un articolo del catalogo prodotti. Ogni `Article` è composto dai seguenti campi:

- `id`: intero che identifica univocamente l'articolo.
- `name`: stringa con il nome dell'articolo.
- `code`: stringa con il codice identificativo.
- `vat`: numero (float) con l'aliquota IVA associata.

Il Listing 6.1 riporta l'implementazione.

```
1 class Article(BaseModel):  
2     id: int  
3     name: str  
4     code: str  
5     vat: float
```

Codice 6.1: Modello Pydantic `Article`.

6.2.1.2 Invoice

Questo modello definisce e valida una fattura con le sue righe di dettaglio. Ogni `Invoice` include:

- `id`: intero identificativo della fattura.
- `customer`: stringa con l'anagrafica cliente.
- `creating_date`: data di emissione.
- `expiring_date`: data di scadenza (opzionale).
- `expected_shipping_date` / `real_shipping_date`: date di spedizione prevista/effectiva (opzionali).
- `total`: numero (float) dell'importo totale.
- `articles`: lista di righe articolo (`ArticleInvoice`).

Il Listing 6.2 riporta il codice del modello.

```
1 class Invoice(BaseModel):
2     id: int
3     customer: str
4     creating_date: date
5     expiring_date: Optional[date] = None
6     expected_shipping_date: Optional[date] = None
7     real_shipping_date: Optional[date] = None
8     total: float
9     articles: List[ArticleInvoice]
```

Codice 6.2: Modello Pydantic Invoice.

6.2.1.3 Job, JobKind, JobStatus

Questi modelli definiscono e validano i job di training/forecasting e il loro ciclo di vita. I campi principali sono:

- `job_id`: stringa identificativa del job.
- `status`: enum con stato corrente (`queued`, `running`, `succeeded`, `failed`).
- `kind`: enum con il tipo di job (`training` o `forecasting`).

- `detail`: stringa opzionale con dettagli/errore.
- `created_at`, `started_at`, `finished_at`: timestamp (stringa) di creazione, avvio e fine (opzionali).

Il Listing 6.3 riporta l'implementazione.

```
1 class JobKind(str, Enum):
2     training = "training"
3     forecasting = "forecasting"
4
5
6 class JobStatus(str, Enum):
7     queued = "queued"
8     running = "running"
9     succeeded = "succeeded"
10    failed = "failed"
11
12
13 class Job(BaseModel):
14     job_id: str
15     status: JobStatus
16     kind: JobKind
17     detail: Optional[str] = None
18     created_at: Optional[str] = None
19     started_at: Optional[str] = None
20     finished_at: Optional[str] = None
```

Codice 6.3: Modelli Pydantic Job/JobKind/JobStatus.

6.2.1.4 NotificationEvent

Questo modello definisce e valida un evento di *Callback / Webhook_G* verso *ErpAI* con stato del job/run:

- `event`: stringa con il tipo di evento notificato.

- `job_id`: stringa identificativa del job.
- `forecast_id`: stringa identificativa della run di previsione.
- `status`: stringa con lo stato del job/run.
- `error`: stringa opzionale con il messaggio di errore.

Il Listing 6.4 riporta il modello di evento.

```
1 class NotificationEvent(BaseModel):
2     event: str
3     job_id: str
4     forecast_id: str
5     status: str
6     error: Optional[str] = None
```

Codice 6.4: Modello Pydantic NotificationEvent.

6.2.1.5 ChromaCollection

Questo modello definisce e valida un documento indicizzato in Chroma:

- `id`: stringa identificativa del documento.
- `content`: stringa con il testo/descrizione indicizzata.
- `metadata`: dizionario di metadati associati.
- `embedding`: lista di numeri (float) per l'*Embedding_G*.

Il Listing 6.5 riporta l'implementazione.

```
1 class ChromaCollection(BaseModel):
2     id: str
3     content: str
4     metadata: Dict[str, Any]
5     embedding: list[float]
```

Codice 6.5: Modello Pydantic ChromaCollection.

6.2.1.6 ShapExplanation

Questo modello definisce e valida la serializzazione dei valori $SHAP_G$:

- **values**: array NumPy di float per i valori $SHAP_G$, convertiti in lista in serializzazione.

Il Listing 6.6 riporta la definizione.

```
1 class ShapExplanation(BaseModel):
2     model_config = ConfigDict(arbitrary_types_allowed=True)
3
4     values: np.ndarray[Any, np.dtype[np.float64]]
5
6     @field_serializer("values")
7     def _ser_np(self, v: np.ndarray[Any, np.dtype[np.float64]]):
8         if v is not None:
9             return v.tolist()
10        return None
```

Codice 6.6: Modello Pydantic ShapExplanation.

6.2.1.7 PredictionResult

Questo modello definisce e valida il payload di risposta con le previsioni future:

- **future_predictions**: dizionario generico delle previsioni future.
- $MAPE_G$: numero (float) con la metrica di errore medio percentuale.

Il Listing 6.7 riporta il payload di risposta.

```
1 class PredictionResult(BaseModel):
2     future_predictions: Dict[Hashable, Any]
3     MAPE: float
```

Codice 6.7: Modello Pydantic PredictionResult.

6.2.1.8 ForecastSearchResult

Questo modello definisce e valida l'esito di una ricerca semantica su previsioni/spiegazioni:

- `predictions`: lista di stringhe con i risultati previsionali.
- `explanations`: lista di stringhe opzionali con le spiegazioni $SHAP_G$.
- `count`: intero con il numero di risultati trovati.
- `fallback`: booleano che indica se è stato usato un percorso di fallback.
- `error`: stringa opzionale con l'eventuale errore riscontrato.

Il Listing 6.8 riporta il modello.

```
1 class ForecastSearchResult(BaseModel):  
2     predictions: List[str] = Field(default_factory=list)  
3     explanations: List[Optional[str]] = Field(default_factory=list)  
4     count: int = 0  
5     fallback: bool = False  
6     error: Optional[str] = None
```

Codice 6.8: Modello Pydantic ForecastSearchResult.

6.2.2 Modelli SQLAlchemy

6.2.2.1 XGBoostPrediction

Questo modello definisce e valida la tabella `predictions_xgboost` per le righe previsionali:

- `forecast_id`: stringa (*Universally Unique Identifier (UUID)_G*) di riferimento alla *Job / Run di previsione_G* di previsione.
- `article_id`: intero identificativo dell'articolo.
- `article_name`: stringa con il nome dell'articolo.

- `prediction_date`: data/ora (datetime) della previsione.
- `predicted_quantity`: numero (float) della quantità stimata.
- `run`: relazione opzionale verso `ForecastRun` (lato N:1).

Il Listing 6.9 riporta la definizione della tabella.

```
1 class XGBoostPrediction(SQLModel, table=True):
2     __tablename__: ClassVar[str] = "predictions_xgboost"
3     __table_args__ = (
4         UniqueConstraint(
5             "forecast_id", "article_id", "prediction_date",
6             ↪ name="uq_pred_key"
7         ),
8     )
9     id: Optional[int] = Field(default=None, primary_key=True)
10
11     forecast_id: str = Field(
12         sa_column=Column(VARCHAR(36), ForeignKey("forecast_runs.id"))
13     )
14     article_id: int = Field(index=True)
15     article_name: str = Field(max_length=255)
16     prediction_date: datetime = Field(index=True)
17     predicted_quantity: float
18
19     # lato N:1
20     run: Optional["ForecastRun"] =
    ↪ Relationship(back_populates="predictions")
```

Codice 6.9: Modello SQLModel XGBoostPrediction.

6.2.2.2 ForecastRun

Questo modello definisce e valida la tabella `forecast_runs`:

- `id`: stringa *UUID_G* identificativa della *Job / Run di previsione_G*.
- `mape`: numero (float) con la metrica di errore della run.
- `created_at`: timestamp (datetime) di creazione.
- `predictions`: lista di `XGBoostPrediction` (relazione 1:N).

Il Listing 6.10 riporta il modello.

```
1 class ForecastRun(SQLModel, table=True):
2     __tablename__: ClassVar[str] = "forecast_runs"
3
4     id: str = Field(sa_column=Column(VARCHAR(36), primary_key=True))
5     mape: float = Field(index=True)
6     created_at: datetime = Field(default_factory=lambda:
7     ↪ datetime.now(timezone.utc))
8
9     predictions: List["XGBoostPrediction"] =
10    ↪ Relationship(back_populates="run")
```

Codice 6.10: Modello `SQLModel ForecastRun`.

6.2.2.3 JobRun (SQLModel)

Questo modello definisce e valida le run dei job persistite in jobs. I campi principali sono:

- `id`: stringa *UUID_G* identificativa della *Job / Run di previsione_G*.
- `kind`: enum `JobKind` che indica il tipo di job.
- `status`: enum `JobStatus` con lo stato corrente.
- `detail`: stringa opzionale con dettagli/errore.
- `dataset_id`: stringa opzionale di riferimento al dataset usato.
- `output_forecast_run_id`: stringa opzionale con la run di previsione prodotta.

- `created_at`: timestamp (datetime) di creazione.
- `started_at`, `finished_at`: timestamp (datetime) di avvio e fine (opzionali).
- `updated_at`: timestamp (datetime) dell'ultimo aggiornamento.

Il Listing 6.11 riporta il modello persistito.

```
1 class JobRun(SQLModel, table=True):
2     __tablename__: ClassVar[str] = "jobs"
3
4     id: str = Field(
5         default_factory=lambda: str(uuid4()),
6         sa_column=Column(VARCHAR(36), primary_key=True),
7     )
8     kind: JobKind = Field(
9         sa_column=Column(Enum(JobKind, name="job_kind"),
10             ↪ nullable=False)
11     )
12     status: JobStatus = Field(
13         default=JobStatus.queued,
14         sa_column=Column(Enum(JobStatus, name="job_status"),
15             ↪ nullable=False),
16     )
17     detail: Optional[str] = Field(default=None, sa_column=Column(Text,
18             ↪ nullable=True))
19     dataset_id: Optional[str] = Field(default=None, index=True,
20             ↪ max_length=128)
21     output_forecast_run_id: Optional[str] = Field(
22         default=None, index=True, max_length=36
23     )
24     created_at: datetime = Field()
```

```

24     default_factory=lambda: datetime.now(timezone.utc), index=True
25 )
26 started_at: Optional[datetime] = None
27 finished_at: Optional[datetime] = None
28 updated_at: datetime = Field(default_factory=lambda:
    ↪ datetime.now(timezone.utc))

```

Codice 6.11: Modello SQLAlchemy JobRun.

6.2.3 Modelli di configurazione

6.2.3.1 Modello di configurazione JWTConfig

Dataclass che definisce e valida la configurazione *JWT_G*:

- `secret_key`: stringa con la chiave segreta per la firma dei token.
- `algorithm`: stringa con l'algoritmo di firma (es. HS256).
- `expire_minutes`: intero con la durata di validità dei token in minuti.

Il Listing 6.12 riporta il dataclass di configurazione.

```

1 @dataclass
2 class JWTConfig:
3     secret_key: str = os.getenv("JWT_SECRET_KEY", "default")
4     algorithm: str = os.getenv("JWT_ALGORITHM", "HS256")
5     expire_minutes: int = int(os.getenv("JWT_EXPIRE_MINUTES", "60"))

```

Codice 6.12: Dataclass JWTConfig.

6.2.3.2 Modello di configurazione XGBoostConfig

La configurazione di XGBoost è definita come dataclass per garantire tipizzazione forte e default espliciti. `XGBoostConfig` raccoglie gli iperparametri chiave:

- `n_estimators`: numero di alberi; più alberi aumentano la capacità ma anche il rischio di overfitting e il tempo di training.

- `learning_rate`: shrinkage per ogni passo; valori bassi rendono il training più stabile ma richiedono più iterazioni per convergere.
- `max_depth`: profondità massima degli alberi; controlla la complessità delle interazioni (profondità elevate catturano non linearità ma aumentano l'overfitting).
- `n_jobs`: core utilizzati nel training; `-1` sfrutta tutti i core disponibili.
- `min_child_weight`: peso minimo in un nodo figlio; evita nodi con poche osservazioni e migliora la robustezza su dati sparsi.
- `subsample`: frazione di righe usate per ciascun albero; introduce bagging stocastico e riduce la varianza.
- `colsample_bytree`: frazione di colonne per albero; diminuisce la correlazione tra alberi e aiuta la generalizzazione.
- `reg_lambda` / `reg_alpha`: regolarizzazioni L2/L1 sui pesi; tengono sotto controllo la magnitudine dei coefficienti e prevenono modelli troppo complessi.
- `random_state`: seme di riproducibilità; rende deterministici split e campionamenti.
- `tree_method` / `max_bin`: algoritmo di costruzione e granularità dell'istogramma; `hist` con `max_bin` regola velocità e approssimazione delle soglie.
- `objective`: funzione obiettivo (regressione `squarederror`); definisce la loss ottimizzata.
- `eval_metric`: metrica di valutazione interna (*`RMSEG`*); guida la diagnostica e l'eventuale early stopping.
- `early_stopping_rounds`: round senza miglioramento prima di fermare il training (0 = disabilitato); evita iterazioni inutili a metrica saturata.

- `target_transform`: trasformazione del target (es. `log1p`); stabilizza la distribuzione e gestisce valori molto variabili.
- `verbosity`: livello di log del training; controlla il dettaglio dell'output.

Il Listing 6.13 riporta la dataclass completa.

```
1 @dataclass
2 class XGBoostConfig:
3     n_estimators: int = 1000
4     learning_rate: float = 0.05
5     max_depth: int = 6
6     n_jobs: int = -1
7     min_child_weight: float = 5.0
8     subsample: float = 0.9
9     colsample_bytree: float = 0.9
10    reg_lambda: float = 1.0
11    reg_alpha: float = 0.0
12    random_state: int = 42
13    tree_method: str = "hist"
14    max_bin: int = 256
15    objective: str = "reg:squarederror"
16    eval_metric: str = "rmse"
17    early_stopping_rounds: int = 50
18    target_transform: str = "log1p"
19    verbosity: int = 1
```

Codice 6.13: Dataclass di configurazione `XGBoostConfig`.

6.3 Servizi

Nella cartella `services/` si trovano le classi che implementano la logica di business e orchestrano il flusso di previsione. Il diagramma 6.2 illustra le principali classi di servizio e le loro interazioni.

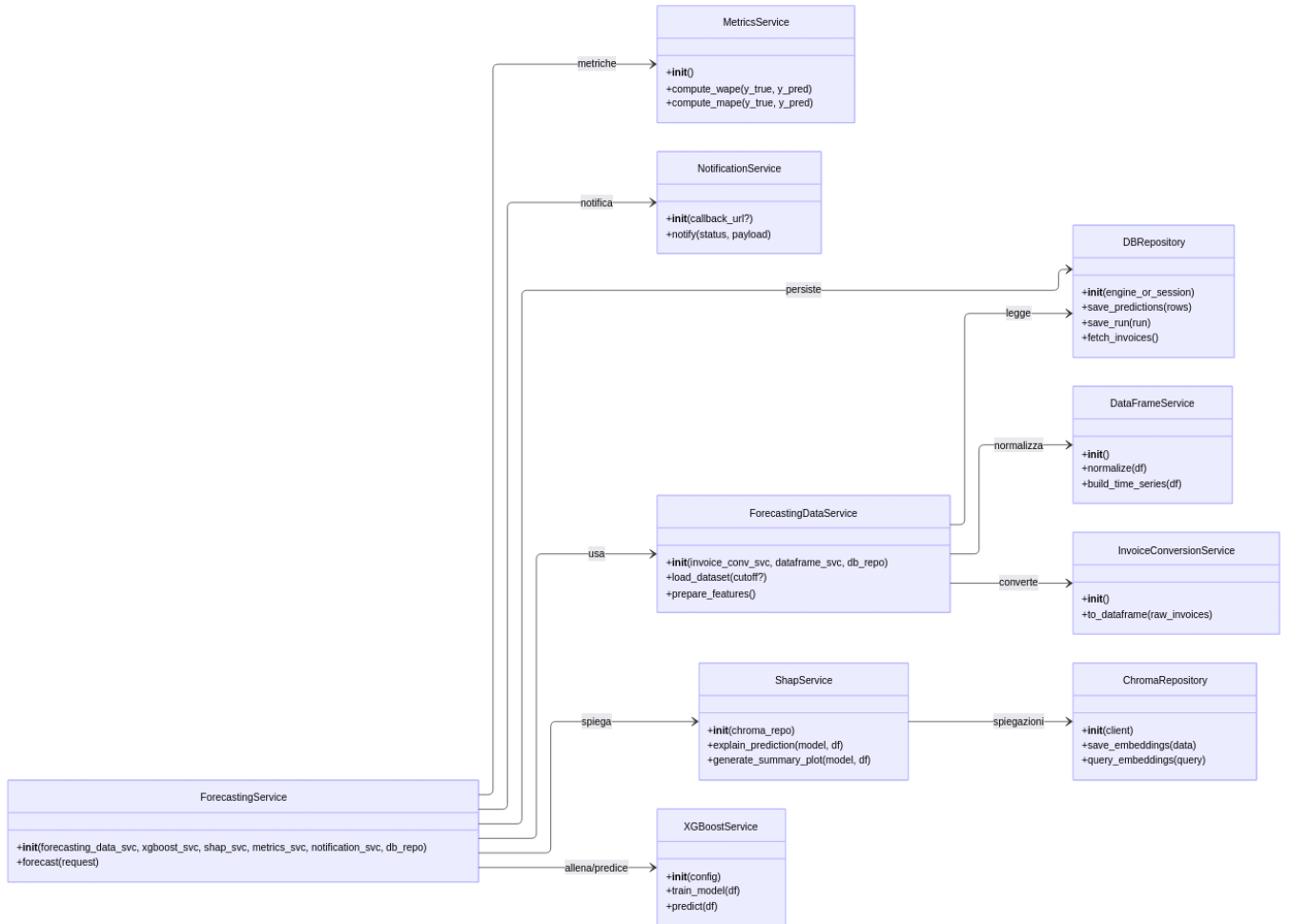


Figura 6.2: Panoramica dei servizi e delle loro interazioni.

6.3.1 InvoiceConversionService

Il servizio `InvoiceConversionService` si occupa di leggere `fact_invoices` dal database DuckDB e restituire un dataframe Pandas pronto per l'addestramento del modello normalizzando tipi, aggregando le fatture per mese e articolo. Il metodo principale `convert_db_to_df` svolge le seguenti operazioni:

- Interrogare il database DuckDB per estrarre `fact_invoices`.
- Convertire le colonne data in formato datetime e normalizzare i tipi numerici.
- Aggregare le righe per mese e articolo sommando le quantità.
- Restituire un dataframe Pandas ordinato.

Il Listing 6.14 riporta l'implementazione del servizio.

```

1 class InvoiceConversionService:
2     """Versione ridotta per la tesi: mostra solo i passaggi chiave."""
3
4     def __init__(self):
5         self.duck: DuckDBRepository = DuckDBRepository()
6
7     def convert_db_data_to_df(self) -> pd.DataFrame:
8         df = self.duck.fetch_fact_invoices()
9
10        # Normalizzazione colonne data e numeriche
11        for col in ("invoice_date", "expiring_date", "shipping_date",
12        ↪ "expected_shipping_date"):
13            if col in df.columns:
14                df[col] = pd.to_datetime(df[col], errors="coerce")
15        for col in ("article_id", "customer_id", "quantity",
16        ↪ "unit_price_discounted", "unit_price_list"):
17            if col in df.columns:
18                df[col] = pd.to_numeric(df[col], errors="coerce")
19
20        # Aggregazione per mese/articolo sommando le quantità
21        if {"invoice_date", "article_id"} <= set(df.columns):
22            df = df.dropna(subset=["invoice_date"]).copy()
23            df["invoice_month"] =
24            ↪ df["invoice_date"].dt.to_period("M").dt.to_timestamp()
25            df = (
26                df.groupby(["article_id", "article_name",
27                ↪ "invoice_month"], dropna=False)
28                .agg({"quantity": "sum"})
29                .reset_index()
30            )
31
32        # Riempimento buchi temporali e flag vendite mensili
33        filled: list[pd.DataFrame] = []

```

```

30         for aid, grp in df.groupby("article_id", sort=False):
31             rng = pd.date_range(grp["invoice_month"].min(),
32                                 ↪ grp["invoice_month"].max(), freq="MS")
33             res = grp.set_index("invoice_month").reindex(rng)
34             res["article_id"] = aid
35             res["article_name"] = res["article_name"].ffill().bfill()
36             ↪ ll().fillna(f"Articolo_{aid}")
37             res["quantity"] = res["quantity"].fillna(0.0)
38             res["sold_this_month"] = (res["quantity"] >
39                                     ↪ 0).astype(int)
40             filled.append(res.reset_index().rename(columns={"index": "invoice_month"}))
41
42         df = pd.concat(filled, ignore_index=True) if filled else
43         ↪ df
44
45     # ...
46     # (Altre colonne/filtri mantenuti nel codice completo non
47     ↪ mostrato)
48     # ...
49
50     cols = ["article_id", "article_name", "invoice_month",
51             ↪ "quantity", "sold_this_month"]
52     existing = [c for c in cols if c in df.columns]
53     return df[existing] if existing else df

```

Codice 6.14: Servizio InvoiceConversionService.

6.3.2 DataFrameService

Il servizio DataFrameService seleziona e prepara le feature numeriche per l'addestramento:

- legge dal config la colonna data e target;
- esclude target, data e `article_id` dall'insieme delle feature;
- opzionalmente include colonne di prezzo e relative derivate;

- offre helper per lo split temporale train/test.

Il Listing 6.15 riporta il codice del servizio.

```
1 class DataFrameService:
2     """Versione ridotta per la tesi: mostra solo i passaggi
3     ↪ principali."""
4
5     PRICE_COLUMNS = [
6         "unit_price_discounted",
7         "unit_price_list",
8         "discount_percent",
9         "total_before_discount",
10        "total_after_discount",
11        "vat_rate",
12    ]
13
14    def __init__(self):
15        self.__config = DataFrameConfig() # date_col, target_col,
16        ↪ exclude
17
18    def get_date_col(self) -> str:
19        return self.__config.date_col
20
21    def get_target_col(self) -> str:
22        return self.__config.target_col
23
24    def get_price_columns(self, include_derivatives: bool = False) ->
25    ↪ list[str]:
26        cols = list(self.PRICE_COLUMNS)
27        if include_derivatives:
28            cols.extend(f"{c}-{s}" for c in self.PRICE_COLUMNS for s in
29                ↪ ("_change_1", "_pct_change"))
30        return cols
```

```

28     def prepare_and_split(
29         self, df: pd.DataFrame, test_size: float = 0.2
30     ) -> tuple[pd.DataFrame, pd.Series, pd.DataFrame, pd.Series,
31         list[str]]:
32         # Pipeline sintetica: verifica target, selezione feature
33         ↪ numeriche, split temporale
34         if self.__config.target_col not in df.columns:
35             raise ValueError(f"Target column
36                 ↪ {self.__config.target_col} not in dataframe.")
37         y = df[self.__config.target_col]
38         feature_columns = self.get_feature_columns(df)
39         X = df[feature_columns].fillna(0)
40
41         sorted_idx = df.sort_values(self.__config.date_col).index
42         X_train, X_test, y_train, y_test = train_test_split(
43             X.loc[sorted_idx], y.loc[sorted_idx], test_size=test_size,
44             ↪ shuffle=False
45         )
46         return X_train, y_train, X_test, y_test, feature_columns
47
48     def prepare_features(self, df: pd.DataFrame) -> pd.DataFrame:
49         if self.__config.target_col not in df.columns:
50             raise ValueError(f"Target column
51                 ↪ {self.__config.target_col} not in dataframe.")
52         feature_columns = self.get_feature_columns(df)
53         return df[feature_columns].fillna(0)
54
55     def get_feature_columns(self, df: pd.DataFrame) -> list[str]:
56         feature_columns = [
57             c
58             for c in df.select_dtypes(include="number").columns
59             if c not in {self.__config.target_col,
60                 ↪ self.__config.date_col, "article_id"}
61         ]

```

```

56         if getattr(self.__config, "exclude", None):
57             feature_columns = [c for c in feature_columns if c not in
58                               ↪ set(self.__config.exclude)]
59         return feature_columns

```

Codice 6.15: Servizio DataFrameService.

6.3.3 ForecastingDataService

Il servizio ForecastingDataService coordina la preparazione dei dati unendo conversione fatture e *Feature engineering*:

- carica il dataframe fatture tramite InvoiceConversionService;
- normalizza/filtra la colonna data, applicando opzionalmente un cutoff temporale;
- delega a DataFrameService la creazione delle feature e restituisce dataset, feature e target;
- espone un helper per generare righe future autoregressive.

Il Listing 6.16 riporta l'implementazione.

```

1  class ForecastingDataService:
2      """Versione ridotta per la tesi: passaggi principali di
3      ↪ preparazione dati."""
4
5      def __init__(
6          self,
7          *,
8          data_frame_service: DataFrameService | None = None,
9          invoice_service: InvoiceConversionService | None = None,
10     ):
11         self._dfs = data_frame_service or DataFrameService()
12         self._invoice_service = invoice_service or
13         ↪ InvoiceConversionService()

```

```

13     def get_data_frame_service(self) -> DataFrameService:
14         return self._dfs
15
16     def get_price_columns(self, include_derivatives: bool = False) ->
17     ↪ list[str]:
18         return self._dfs.get_price_columns(include_derivatives=include_
19         ↪ e_derivatives)
20
21     def load_fact_df(self, cutoff_date: pd.Timestamp | None = None) ->
22     ↪ pd.DataFrame:
23         df = self._invoice_service.convert_db_data_to_df()
24         date_col = self._dfs.get_date_col()
25         df[date_col] = pd.to_datetime(df[date_col], errors="coerce")
26         df = df.dropna(subset=[date_col])
27         if cutoff_date is not None:
28             df = df[df[date_col] <
29             ↪ pd.to_datetime(cutoff_date).tz_localize(None)]
30         return df
31
32     def prepare_regression_frame(
33     self, cutoff_date: pd.Timestamp | None = None
34     ) -> tuple[pd.DataFrame, list[str], str, str]:
35         raw = self.load_fact_df(cutoff_date=cutoff_date)
36         date_col = self._dfs.get_date_col()
37         target_col = self._dfs.get_target_col()
38         prepared =
39         ↪ self._dfs.prepare_features(raw).sort_values(date_col)
40         feature_cols = self._dfs.get_feature_columns(prepared)
41         return prepared, feature_cols, date_col, target_col
42
43     def prepare_dataset(
44     self, cutoff_date: pd.Timestamp | None = None
45     ) -> tuple[pd.DataFrame, list[str], str, str]:
46         return self.prepare_regression_frame(cutoff_date=cutoff_date)

```

```

42
43     def build_future_rows(
44         self,
45         hist_df: pd.DataFrame,
46         feature_columns: list[str],
47         future_dates: pd.DatetimeIndex,
48     ) -> pd.DataFrame:
49         return self._dfs.build_future_rows_autoregressive(
50             hist_df, feature_columns, future_dates
51         )
52
53     # ...
54     # (Altre funzioni di logging/copertura mensile omesse nella
55     ↪ versione ridotta)
56     # ...

```

Codice 6.16: Servizio ForecastingDataService.

6.3.4 MetricsService

La classe `MetricsService` incapsula le metriche di valutazione personalizzate usate per l'addestramento e la diagnosi del modello XGBoost:

- `mape_percent`: Mean Absolute Percentage Error ($MAPE_G\%$).
- `smape_percent`: Symmetric MAPE (*Symmetric Mean Absolute Percentage Error* ($SMAPE)_G\%$).
- `wape_percent`: Weighted Absolute Percentage Error ($WAPE_G\%$).

Il Listing 6.17 riporta il servizio.

```

1 class MetricsService:
2     def mape_percent(
3         self, y_true: np.ndarray, y_pred: np.ndarray, eps: float =
4         ↪ 1e-7
5     ) -> float:

```

```

5         denom = np.where(np.abs(y_true) < eps, eps, y_true)
6         return float(np.mean(np.abs((y_true - y_pred) / denom)) *
           ↪ 100.0)
7
8     def smape_percent(
9         self, y_true: np.ndarray, y_pred: np.ndarray, eps: float =
           ↪ 1e-7
10    ) -> float:
11        denom = np.maximum(eps, (np.abs(y_true) + np.abs(y_pred)) /
           ↪ 2.0)
12        return float(np.mean(np.abs(y_true - y_pred) / denom) * 100.0)
13
14    def wape_percent(
15        self, y_true: np.ndarray, y_pred: np.ndarray, eps: float =
           ↪ 1e-7
16    ) -> float:
17        den = max(eps, float(np.sum(np.abs(y_true))))
18        return float(np.sum(np.abs(y_true - y_pred)) / den * 100.0)
19

```

Codice 6.17: Servizio MetricsService.

6.3.5 NotificationService

Il servizio NotificationService invia *Callback / Webhook_G HTTP_G* verso il sistema esterno (Laravel) al termine dei job/run:

- estrae le credenziali e l'endpoint dalle variabili d'ambiente;
- ottiene un token *JWT_G* con una richiesta di login e lo usa per autenticare la chiamata di notifica;
- accetta payload NotificationEvent o dizionari e li invia in POST all'endpoint di *Callback / Webhook_G*.

Il Listing 6.18 riporta l'implementazione del servizio.


```
1 class NotificationService:
2     def __init__(self):
3         self._notify_env = "LARAVEL_NOTIFY_URL"
4
5     def _get_token(self) -> Optional[str]:
6         url = os.getenv("LARAVEL_LOGIN_URL")
7         email = os.getenv("LARAVEL_MAIL")
8         password = os.getenv("LARAVEL_PASSWORD")
9         if not url or not email or not password:
10             return None
11         r = requests.post(url, json={"email": email, "password":
12             ↪ password}, timeout=5)
13         r.raise_for_status()
14         data = (
15             r.json()
16             if r.headers.get("content-type",
17             ↪ "").startswith("application/json")
18             else {}
19         )
20         return data.get("token")
21
22     def notify(self, payload: NotificationEvent | dict[str, Any]) ->
23     ↪ None:
24         url = os.getenv(self._notify_env)
25         if not url:
26             return
27         data = payload.model_dump() if isinstance(payload,
28             ↪ NotificationEvent) else payload
29         token = self._get_token()
30         headers = {"Authorization": f"Bearer {token}"} if token else
31             ↪ None
32         requests.post(url, json=data, headers=headers, timeout=5)
```

Codice 6.18: Servizio NotificationService.

6.3.6 DataNormalizeService

Il servizio `DataNormalizeService` è pensato per rendere le previsioni e le spiegazioni comprensibili al Modulo *LLM_G*, trasformando numeri e *SHAP_G* in linguaggio naturale e documenti indicizzabili.

Esempi di input/output:

- **Input:** previsioni XGBoost (`article_id`, `prediction_date`, `predicted_quantity`) e valori *SHAP_G* per ciascuna riga.
- **Output testo (predizioni):** frasi come “Per l’articolo A123, il 01 marzo 2025 prevediamo 12.4 unità (finestra 1/3).”
- **Output testo (spiegazioni):** frasi come “Spiegazione per A123 (marzo 2025): feature prezzo scontato ha aumentato la previsione, stagionalità ha ridotto la previsione.”
- **Documenti embedding:** blocchi di testo con contesto (articolo, periodo, quantità, top driver *SHAP_G*) e metadati (`article_id`, `data`, `forecast_window`).

Questi output alimentano sia la presentazione all’utente, sia la creazione di *Embedding_G* (ChromaDB) per recupero semantico e interazione con l’*LLM_G*.

6.3.7 XgboostService

La classe `XgboostService` incapsula configurazione, training e predizione del modello XGBoost:

- metriche di valutazione personalizzate gestite dal servizio `MetricsService` (sez. 6.3.4);
- servizio dati per preparare feature e target (`ForecastingDataService`, sez. 6.3.3);
- parametri del modello (`XGBoostConfig`) per controllare alberi, regolarizzazione e sampling.

6.3.7.1 Inizializzazione

Il costruttore crea il regressore XGBoost con gli iperparametri di `XGBoostConfig`, collega il servizio dati e le metriche e inizializza gli stati interni (feature usate, ultima metrica calcolata). Il Listing 6.19 riporta l’inizializzazione della classe.

```

1 class XGBoostService:
2
3     def __init__(
4         self,
5         *,
6         metrics: MetricsService | None = None,
7         data_service: ForecastingDataService | None = None,
8         config: XGBoostConfig | None = None,
9     ):
10         self.__cfg = config or XGBoostConfig()
11         self.__data_service = data_service or ForecastingDataService()
12         self.__metrics = metrics or MetricsService()

```

Codice 6.19: Inizializzazione di `XGBoostService`.

Il servizio gestisce due processi principali: **training** (prepara i dati e addestra il modello) e **previsione** (genera le quantità future).

6.3.7.1.1 Training.

Il Listing 6.20 riporta il metodo `train_model`. Recupera i dati dal `ForecastingDataService` e addestra il modello: `__fit_model` imputa i missing (mediana/zero), forza il target a valori non negativi, effettua uno split temporale 80/20, applica l’eventuale `log1p` sul target e allena XGBoost. Se disponibile un *Hold-out_G*, calcola il *WAPE_G%* (invertendo la trasformazione) e lo salva in `__last_wape` per la diagnostica.

```

1 def __prepare_training_data(
2     self, cutoff_date: pd.Timestamp | None
3 ) -> tuple[pd.DataFrame, list[str], str, str]:

```

```

4      """
5      Prepara il dataset di regressione: carica dati, applica cutoff
        ↪ temporale,
6      restituisce frame ordinato, feature, colonna data e target.
7      """
8      prepared, feature_cols, date_col, target_col = (
9          self.__data_service.prepare_regression_frame(cutoff_date=cutoff_date,
        ↪ ff_date)
10     )
11     if prepared.empty:
12         raise ValueError("Nessun dato disponibile per
        ↪ l'addestramento.")
13     return prepared.sort_values(date_col), feature_cols, date_col,
        ↪ target_col
14
15
16 def train_model(self, cutoff_date: pd.Timestamp | None = None) ->
        ↪ None:
17     """
18     Flusso di training semplificato:
19     1. prepara dati e feature;
20     2. imputazione + split temporale 80/20;
21     3. fit del modello XGBoost;
22     4. calcolo WAPE% su hold-out (se presente).
23     """
24     prepared, feature_cols, date_col, target_col =
        ↪ self.__prepare_training_data(
25         cutoff_date
26     )
27     self.__fit_model(
28         prepared=prepared,
29         feature_cols=feature_cols,
30         date_col=date_col,
31         target_col=target_col,

```

32)

Codice 6.20: Metodo `train_model` di `XGBoostService`.

6.3.7.2 Previsione

Il Listing 6.21 riporta il metodo `predict`. Per generare le previsioni il servizio:

- riaddestra il modello sui dati disponibili fino al mese prima di `start_date`;
- crea le date future e costruisce le feature in modo autoregressivo;
- predice le quantità future e restituisce il `PredictionResult` con la $MAPE_G$ stimata, la matrice delle feature future e la lista delle colonne usate.

```

1 def predict(
2     self,
3     *,
4     start_date: datetime,
5     periods: int,
6     use_actual_if_available: bool = False,
7 ) -> tuple[PredictionResult, np.ndarray, list[str]]:
8     """
9     Flusso di previsione:
10    1. addestra il modello fino al mese precedente a start_date;
11    2. genera date future e feature autoregressive;
12    3. produce quantità previste (o reali se richiesto);
13    4. restituisce PredictionResult, matrice feature e lista colonne.
14    """
15    start_ts = self.__as_naive_timestamp(start_date).to_period("M").to_timestamp()
16    self.train_and_test_model(cutoff_date=start_ts)
17
18    prepared_all, _, _, _ =
19    self.__data_service.prepare_regression_frame()

```

```

19     preds_df, X_pred = self.__build_real_predictions(
20         prepared=prepared_all,
21         start_ts=start_ts,
22         periods=periods,
23         use_actual_if_available=use_actual_if_available,
24     )
25
26     mape_value = self.__metrics.clean(self.__last_wape) if
    ↪ self.__last_wape is not None else None
27     return (
28         PredictionResult(
29             future_predictions=preds_df.to_dict(),
30             MAPE=mape_value if mape_value is not None else 99.9,
31         ),
32         X_pred,
33         list(self.feature_columns or []),
34     )

```

Codice 6.21: Metodo predict di XGBoostService.

6.3.8 ShapService

Il servizio ShapService fornisce spiegazioni locali e visualizzazioni delle feature per il modello XGBoost:

- `explain_prediction` calcola i valori $SHAP_G$ su un set di input tramite `TreeExplainer`.
- `generate_summary_plot` produce il summary plot principale e restituisce il path salvato.

Il Listing 6.22 riporta l'estrazione dei valori $SHAP_G$, mentre il Listing 6.23 genera il summary plot.

```

1 def explain_prediction(
2     self, X: np.ndarray[Any, np.dtype[np.float64]], model: Any

```

```

3 ) -> ShapExplanation:
4     """
5     Calcola i valori SHAP per le feature del modello.
6     1. costruisce un TreeExplainer sul modello XGBoost;
7     2. applica l'explainer alle righe di input;
8     3. restituisce i valori SHAP incapsulati in ShapExplanation.
9     """
10    explainer = shap.TreeExplainer(model)
11    explanation = explainer(X)
12    shap_response =
13    ↪ ShapExplanation(values=np.array(explanation.values))
14    return shap_response

```

Codice 6.22: Metodo explain_prediction di ShapService.

```

1 def generate_summary_plot(
2     self,
3     shap_explanation: ShapExplanation,
4     *,
5     features: np.ndarray[Any, np.dtype[np.float64]],
6     feature_names: list[str],
7     output_dir: Path,
8 ) -> dict[str, str]:
9     """
10    Genera il summary plot SHAP e restituisce il path del file
11    ↪ salvato.
12    """
13
14    plots: dict[str, str] = {}
15
16    summary_plot_path = output_dir / "shap_summary_plot.png"
17    summary_path = self.save_summary_plot(
18        shap_explanation,
19        features=features,
20        feature_names=feature_names,
21        output_path=summary_plot_path,

```

```
20     )
21     if summary_path is not None:
22         plots["summary_plot"] = str(summary_path)
23
24     return plots
```

Codice 6.23: Metodo `generate_summary_plot` di `ShapService`.

6.3.9 ForecastingService

La classe `ForecastingService` fa da orchestratore principale per il flusso di previsione. Il metodo chiave è `forecast` e coordina l'intero processo di previsione:

- invoca `XGBoostService.predict` per ottenere previsioni future e feature generate;
- arricchisce il payload con feature usate, $MAPE_G$ stimata, data finale;
- traduce e prepara le previsioni per salvataggio ed esposizione API_G ;
- salva i risultati tramite `ForecastPersistenceService` su database/ChromaDB;
- calcola le spiegazioni $SHAP_G$ e genera i plot di interpretabilità.

Il Listing 6.24 riporta il metodo `forecast`.

```
1 class ForecastingService:
2     def __init__(
3         self,
4         *,
5         xgb: XGBoostService | None = None,
6         shap: ShapService | None = None,
7         chroma_repo: ChromaRepository | None = None,
8         db_repo: DBRepository | None = None,
9         normalize: DataNormalizeService | None = None,
10    ):
```



```

11     self.__xgboost_service = xgb or XGBoostService()
12     self.__shap_service = shap or ShapService()
13     self.__chroma_repository = chroma_repo or ChromaRepository()
14     self.__db_repository = db_repo or DBRepository()
15     self.__normalize_service: DataNormalizeService = (
16         normalize or DataNormalizeService()
17     )
18     self.__persistence = ForecastPersistenceService(
19         chroma_repo=self.__chroma_repository,
20         ↪ db_repo=self.__db_repository
21     )
22
23 def forecast(
24     self,
25     *,
26     forecast_id: str,
27     start_date: datetime,
28     periods: int,
29 ) -> Dict[str, Any]:
30     results, X_pred, feature_columns =
31         ↪ self.__xgboost_service.predict(
32             start_date=start_date, periods=periods
33         )
34
35     shap_explanation = self.__shap_service.explain_prediction(
36         X_pred, self.__xgboost_service.get_model()
37     )
38
39     normalizer = self.__normalize_service
40
41     translated_predictions =
42         ↪ normalizer.translate_multiple_predictions_to_text(
43             results

```

```

42         )
43
44         translated_explanations =
45             ↪ normalizer.translate_multiple_explanations_to_text(
46                 shap_explanation, feature_columns, results
47             )
48
49         context_documents, context_metadatas =
50             ↪ normalizer.build_embedding_documents(
51                 results, shap_explanation, feature_columns
52             )
53
54         data: Dict[str, Any] = {
55             "forecast_id": forecast_id,
56             "predictions": results,
57             "start_date": start_date.isoformat(),
58             "frequency": "M",
59             "periods": periods,
60         }
61
62         df = pd.DataFrame.from_dict(results.future_predictions)
63         data["predictions_df"] = df.to_dict(orient="records")
64
65         self.__persistence.save(
66             translated_predictions=translated_predictions,
67             translated_explanations=translated_explanations,
68             context_documents=context_documents,
69             context_metadatas=context_metadatas,
70             forecast_id=forecast_id,
71             db_payload=data,
72         )
73
74         return data

```

Codice 6.24: Metodo forecast di ForecastingService.

6.4 API Router

Nella cartella `api/` si trovano i router FastAPI che espongono gli endpoint `RESTG` per interagire con il modulo di previsione, coperti da autenticazione `JWTG` tramite le dipendenze definite nel modulo `jwt_dependencies`. Per completezza si include anche il router di autenticazione dell'`auth-server`, che è un modulo separato rispetto al servizio di forecasting ma fornisce le chiavi pubbliche e i token necessari.

6.4.1 Authentication (Auth Server)

Router dell'`auth-server`, modulo autonomo che rilascia token `JWTG` e pubblica il `JWKSG` usato dagli altri servizi:

- GET `/.well-known/jwks.json`: restituisce il set di chiavi pubbliche in formato `JWKSG` tramite `JWKSService`.
- POST `/token`: espone la dependency `jwt_dependencies.get_new_token` per generare un nuovo `JWTG` firmato.

Il Listing 6.25 riporta il router.

```
1 authentication = APIRouter()
2
3
4 @authentication.get("/.well-known/jwks.json")
5 async def get_public_keys() -> JWKS:
6     return JWKSService().get_jwks()
7
8
9 @authentication.post("/token")
10 async def get_token(token: JWT =
    ↳ Depends(jwt_dependencies.get_new_token)) -> JWT:
11     return token
```

Codice 6.25: Router di autenticazione: `/.well-known/jwks.json`, `/token` (modulo `auth-server`).

6.4.2 /verify-token

Router minimale per verificare i token JWT_G in ingresso:

- endpoint POST /verify-token che accetta le credenziali via dependency injection e restituisce un semplice messaggio di conferma.
- sfrutta `jwt_dependencies.check_token` per bloccare le richieste non autorizzate.

Il Listing 6.26 riporta il router.

```
1 jwt = APIRouter()
2
3 @jwt.post("/verify-token")
4 def token_verification(payload: dict =
  ↳ Depends(jwt_dependencies.check_token)):
5     return {"msg": "token ok"}
```

Codice 6.26: Router /verify-token.

6.4.3 /forecast

L'endpoint di previsione avvia in background una run di forecasting e restituisce subito l'identificativo del job:

- POST /forecast: riceve `start_date` e `periods`, crea un Job di tipo `forecasting` e imposta l'header `Location` verso `/jobs/{id}`.
- avvia un task asincrono che aggiorna lo stato del job (`queued` → `running` → `succeeded/failed`), invoca `ForecastingService.forecast` e invia notifiche esterne tramite `NotificationService`.
- in caso di errore durante il task, marca il job come `failed` e spedisce una notifica con il dettaglio dell'errore.

Il Listing 6.27 riporta il router.

```

1 forecasting =
  ↳ APIRouter(dependencies=[Depends(jwt_dependencies.check_token)])
2
3 class ForecastRequest(BaseModel):
4     start_date: datetime = Field(..., description="Data di inizio
  ↳ previsione inclusa")
5     periods: int = Field(default=6, gt=0, description="Numero di
  ↳ finestre da prevedere")
6
7     @model_validator(mode="after")
8     def _check_dates(self) -> "ForecastRequest":
9         if self.periods <= 0:
10             raise ValueError("periods must be greater than zero")
11         return self
12
13 @forecasting.post("/forecast")
14 def forecast(
15     background_tasks: BackgroundTasks,
16     response: Response,
17     payload: ForecastRequest,
18 ):
19     job_service = JobService()
20
21     job = job_service.create_job(kind=JobKind.forecasting)
22
23     job_id_raw = getattr(job, "id", None) or getattr(job, "job_id",
  ↳ None)
24     job_id_uuid = UUID(str(job_id_raw))
25     response.headers["Location"] = f"/jobs/{job_id_uuid}"
26
27     forecast_id = str(uuid4())
28
29     def task():
30         try:

```

```
31         job_service.update_job_status(job_id_uuid,
32             ↪ JobStatus.running)
33
34         ForecastingService().forecast(
35             forecast_id=forecast_id,
36             start_date=payload.start_date,
37             periods=payload.periods,
38         )
39
40         job_service.update_job_status(job_id_uuid,
41             ↪ JobStatus.succeeded)
42
43         NotificationService().notify(
44             NotificationEvent(
45                 event="forecast_completed",
46                 job_id=str(job_id_uuid),
47                 forecast_id=forecast_id,
48                 status="succeeded",
49             )
50         )
51
52     except Exception as e:
53
54         job_service.update_job_status(job_id_uuid,
55             ↪ JobStatus.failed, detail=str(e))
56
57         NotificationService().notify(
58             NotificationEvent(
59                 event="forecast_failed",
60                 job_id=str(job_id_uuid),
61                 forecast_id=forecast_id,
62                 status="failed",
63                 error=str(e),
64             )
65         )
66
67     background_tasks.add_task(task)
68
69     return {
70         "job": Job(
```

```

62         job_id=str(job_id_uuid), kind=JobKind.forecasting,
           ↪ status=JobStatus.queued
63     ),
64     "forecast_id": forecast_id,
65 }

```

Codice 6.27: Router /forecast.

6.4.4 /jobs

Router per interrogare lo stato dei job di training/forecasting:

- GET /jobs: ritorna la lista dei job con filtri opzionali kind e status.
- GET /jobs/{job_id}: restituisce il singolo job oppure 404 se non esiste.
- la logica di persistenza e filtro è delegata a JobService.

Il Listing 6.28 riporta il router.

```

1  job = APIRouter(dependencies=[Depends(jwt_dependencies.check_token)])
2
3  @job.get("/jobs/{job_id}", response_model=Job)
4  def get_job(job_id: UUID):
5      job = JobService().get_job(job_id)
6      if not job:
7          raise HTTPException(status_code=404, detail="Job not found")
8      return job
9
10 @job.get("/jobs", response_model=list[Job])
11 def list_jobs(kind: Optional[JobKind] = None, status:
   ↪ Optional[JobStatus] = None):
12     return JobService().list_jobs(kind, status)

```

Codice 6.28: Router /jobs.

6.4.5 /context

Endpoint per la ricerca semantica su previsioni ed *Embedding_G* *SHAP_G* indicizzati in ChromaDB:

- GET /context: accetta una domanda naturale (question), il numero di risultati desiderati e, opzionalmente, un forecast_id per filtrare.
- inizializza pigramente un singleton di ChromaSearchService, esegue query_forecast e restituisce predizioni/spiegazioni rilevanti.
- in caso di eccezione solleva un HTTPException 500 con il dettaglio dell'errore.

Il Listing 6.29 riporta il router.

```
1 chromadb =
  ↳ APIRouter(dependencies=[Depends(jwt_dependencies.check_token)])
2 _search_service = None
3
4 def _get_service() -> ChromaSearchService:
5     global _search_service
6     if _search_service is None:
7         print("[context] Initializing ChromaSearchService singleton",
8             ↳ flush=True)
9         _search_service = ChromaSearchService()
10    return _search_service
11
12 @chromadb.get("/context")
13 async def get_context(question: str, result_count: int = 5,
14     ↳ forecast_id: str = None):
15     try:
16         service = _get_service()
17         resp = service.query_forecast(question, result_count,
18             ↳ forecast_id)
19         print(
20             f"[context] query='{question}' forecast_id={forecast_id}
21             ↳ results_pred={len(resp.predictions)}",
22             flush=True,
23         )
24     return resp
```



```
21     except Exception as e:
22         import traceback
23
24         traceback.print_exc()
25     raise HTTPException(status_code=500, detail=f"Context
    ↪ retrieval error: {e}")
```

Codice 6.29: Router /context.

Capitolo 7

Testing e Continuous Integration

Durante l'ultimo periodo dello stage mi sono occupato di implementare una suite di test automatizzati e una pipeline di Continuous Integration per il progetto. Nella cartella `tests/` del *repository* sono stati creati diversi test per coprire le funzionalità principali del sistema. I test si occupano di:

- Garantire la correttezza delle funzionalità principali del sistema.
- Validare le integrazioni con librerie esterne e servizi di terze parti.
- Prevenire regressioni durante lo sviluppo di nuove funzionalità.

I test si sono divisi in unit test e integration tests, eseguiti tramite il *framework* `pytest`.

7.1 Unit Test

Gli unit test verificano il comportamento di singole funzioni o classi in isolamento, utilizzando mock e stub per simulare dipendenze esterne. Questi test sono veloci da eseguire e forniscono un *feedback* immediato durante lo sviluppo. Sono stati effettuati test per tutto il codice raggiungendo una copertura del 85%.

7.1.1 Esempio

Il file `test_forecasting_service.py` isola la logica del servizio di forecasting sostituendo tutte le dipendenze con stub:

- *Happy path_G*: con finti `XGBoostService`, `ShapService` e `DataNormalizeService` verifica che vengano invocati con i parametri attesi e che il payload persistito contenga predizioni, spiegazioni e documenti per ChromaDB/DB.
- **Errore di serializzazione**: forza un errore nella costruzione del `DataFrame` di output e controlla che il metodo gestisca l'eccezione senza restituire la chiave `predictions_rows`.

Il Listing 7.1 riporta il test.

```
1 class FakeXGB:
2     def __init__(self):
3         self.pred_called = None
4         self.model = object()
5
6     def predict(self, *, start_date, periods):
7         self.pred_called = (start_date, periods)
8         results = SimpleNamespace(
9             future_predictions={
10                 "prediction_date": {0: "2024-02-01", 1: "2024-02-08"},
11                 "y": {0: 1, 1: 2},
12             },
13             other="ok",
14         )
15         X_pred = [[0.1, 0.2]]
16         features = ["f1", "f2"]
17         return results, X_pred, features
18
19     def get_model(self):
20         return self.model
21
22
23 class FakeShap:
24     def __init__(self):
```

```
25     self.called = None
26
27     def explain_prediction(self, X, model):
28         self.called = (tuple(map(tuple, X)), model)
29         return SimpleNamespace(values=[[0.1, -0.1]])
30
31
32 class FakeNormalize:
33     def __init__(self):
34         self.pred_in = None
35         self.expl_in = None
36         self.ctx_called = False
37
38     def translate_multiple_predictions_to_text(self, results):
39         self.pred_in = results
40         return ["p1", "p2"]
41
42     def translate_multiple_explanations_to_text(self, shap_expl,
43 ↪ feature_cols, results):
44         self.expl_in = (shap_expl, tuple(feature_cols), results)
45         return ["e1"]
46
47     def build_embedding_documents(self, prediction_result, shap_expl,
48 ↪ feature_cols):
49         self.ctx_called = True
50         return ["doc"], [{"article_id": "10"}]
51
52 class FakeChromaRepo:
53     pass
54
55 class FakeDBRepo:
56     pass
```

```
57
58
59 class FakePersistence:
60     def __init__(self):
61         self.saved = None
62
63     def save(
64         self,
65         *,
66         translated_predictions,
67         translated_explanations,
68         context_documents,
69         context_metadatas,
70         forecast_id,
71         db_payload,
72     ):
73         self.saved = (
74             translated_predictions,
75             translated_explanations,
76             context_documents,
77             context_metadatas,
78             forecast_id,
79             db_payload,
80         )
81
82
83 def make_service():
84     fs = ForecastingService(
85         xgb=cast(XGBoostService, FakeXGB()),
86         shap=cast(ShapService, FakeShap()),
87         chroma_repo=cast(ChromaRepository, FakeChromaRepo()),
88         db_repo=cast(DBRepository, FakeDBRepo()),
89         normalize=cast(DataNormalizeService, FakeNormalize()),
90     )
```

```
91     setattr(fs, "_ForecastingService__persistence", FakePersistence())
92     return fs
93
94
95 def test_forecast_happy_path_builds_payload_and_persists():
96     fs = make_service()
97     out = fs.forecast(
98         forecast_id="fid",
99         start_date=datetime(2024, 2, 1),
100         periods=2,
101     )
102     xgb = getattr(fs, "_ForecastingService__xgboost_service")
103     assert xgb.pred_called == (datetime(2024, 2, 1), 2)
104     shap = getattr(fs, "_ForecastingService__shap_service")
105     assert shap.called[1] is xgb.get_model()
106     norm = getattr(fs, "_ForecastingService__normalize_service")
107     assert norm.pred_in is not None and norm.expl_in is not None and
108         ↪ norm.ctx_called
109     saved = getattr(fs, "_ForecastingService__persistence").saved
110     assert saved is not None and saved[4] == "fid"
111     assert saved[0] == ["p1", "p2"] and saved[1] == ["e1"]
112     assert saved[2] == ["doc"] and saved[3] == [{"article_id": "10"}]
113     assert out["forecast_id"] == "fid"
114     assert out["start_date"].startswith("2024-02-01")
115     assert out["frequency"] == "2W"
116     assert out["periods"] == 2
117     assert out["predictions"].future_predictions == {
118         "prediction_date": {0: "2024-02-01", 1: "2024-02-08"},
119         "y": {0: 1, 1: 2},
120     }
121     rows = out.get("predictions_rows")
122     assert rows and
123         ↪ rows[0]["prediction_date"].startswith("2024-02-01")
124     assert rows[1]["y"] == 2
```

```
123     assert out.get("end_date").startswith("2024-02-08")
124
125
126 def test_forecast_handles_df_build_error(monkeypatch):
127     fs = make_service()
128
129     def boom(*a, **k):
130         raise ValueError("bad")
131
132     monkeypatch.setattr("pandas.DataFrame.from_dict", boom)
133     out = fs.forecast(
134         forecast_id="fid",
135         start_date=datetime(2024, 2, 1),
136         periods=1,
137     )
138     assert "predictions_rows" not in out
```

Codice 7.1: Unit test del servizio di forecasting.

7.2 Integration Tests

I test di integrazione verificano che i vari componenti del sistema funzionino correttamente insieme. Questi test sono più lenti da eseguire rispetto agli unit test, ma sono fondamentali per garantire che le interazioni tra i moduli siano corrette. Sono stati creati test per le principali funzionalità del sistema, come l'interazione con il database e le chiamate *API_G*.

7.2.1 Esempio

Il test `test_chromadb_api.py` valida l'endpoint `/context` simulando la dipendenza *JWT_G* e il servizio di ricerca ChromaDB:

- usa `app.dependency_overrides` per bypassare la verifica del token con un principal fittizio;

- sostituisce `_get_service` con uno stub per controllare il risultato e tracciare i parametri ricevuti;
- verifica sia il caso di successo (200 con predizioni) sia l'errore server (500) propagato da un servizio che solleva eccezione.

Il Listing 7.2 riporta il test.

```
1 @pytest.mark.asyncio
2 async def test_chromadb_context_success(monkeypatch, client, app):
3
4     class FakeService:
5         def query_forecast(
6             self, question: str, n_results: int = 5, forecast_id: str
7             ↪ | None = None
8         ):
9             return SimpleNamespace(
10                 predictions=["p1", "p2"],
11                 explanations=[None, None],
12                 count=2,
13                 fallback=False,
14             )
15
16     app.dependency_overrides[jwt_deps.jwt_dependencies.check_token] =
17     ↪ lambda: {
18         "sub": "test"
19     }
20
21     monkeypatch.setattr(chroma_api, "_get_service", lambda:
22     ↪ FakeService())
23
24     r = await client.get("/context", params={"question": "abc",
25     ↪ "result_count": 2})
26
27     assert r.status_code == 200
28
29     data = r.json()
30
31     assert data["count"] == 2 and data["predictions"] == ["p1", "p2"]
```



```

24
25     app.dependency_overrides.pop(jwt_deps.jwt_dependencies.check_token)
26     ↪     n, None)
27
28 @pytest.mark.asyncio
29 async def test_chromadb_context_error(monkeypatch, client, app):
30
31     class BoomService:
32         def query_forecast(self, *a, **k):
33             raise RuntimeError("boom")
34
35     app.dependency_overrides[jwt_deps.jwt_dependencies.check_token] =
36     ↪     lambda: {
37         "sub": "test"
38     }
39
40     monkeypatch.setattr(chroma_api, "_get_service", lambda:
41     ↪     BoomService())
42
43
44     r = await client.get("/context", params={"question": "abc"})
45     assert r.status_code == 500
46
47
48 @pytest.mark.asyncio
49 async def test_chromadb_context_with_forecast_id(monkeypatch, client,
50 ↪     app):
51
52     class FakeService:
53         def __init__(self):
54             self.last = None
55
56         def query_forecast(
57             self, question: str, n_results: int = 5, forecast_id: str
58             ↪     | None = None

```

```
53         ):
54             self.last = (question, n_results, forecast_id)
55             return SimpleNamespace(
56                 predictions=[], explanations=[], count=0,
57                 ↪ fallback=False
58             )
59
60     svc = FakeService()
61     app.dependency_overrides[jwt_deps.jwt_dependencies.check_token] =
62     ↪ lambda: {
63         "sub": "test"
64     }
65     monkeypatch.setattr(chroma_api, "_get_service", lambda: svc)
66
67     fid = "abc-123"
68     r = await client.get(
69         "/context", params={"question": "q", "result_count": 1,
70         ↪ "forecast_id": fid}
71     )
72     assert r.status_code == 200
73     assert svc.last == ("q", 1, fid)
74     app.dependency_overrides.pop(jwt_deps.jwt_dependencies.check_token)
75     ↪ n, None)
76
77     app.dependency_overrides.pop(jwt_deps.jwt_dependencies.check_token)
78     ↪ n, None)
```

Codice 7.2: Integration test per l'endpoint /context

7.3 Continuous Integration

Nella pipeline di Continuous Integration la suite di test viene eseguita automaticamente su ogni push (tutti i branch), sulle pull request verso `develop` e può essere avviata manualmente. Il job principale gira su un runner `archlinux al-`

l'interno di un container Python 3.12 pubblicato internamente, così da replicare lo stesso ambiente usato in sviluppo e in staging.

- Il caching di `pip` riduce i tempi di installazione usando come chiave l'hash dei `requirements.txt`.
- Le dipendenze vengono installate per tutti i moduli (`auth-server`, `duckdb`, `forecasting`) più i pacchetti test comuni (`pytest`, `pytest-asyncio`, `anyio`, `pytest-cov`).
- L'esecuzione dei test usa database SQLite in memoria per evitare side effect esterni e genera il report di copertura in formato *eXtensible Markup Language (XML)*_G/*HyperText Markup Language (HTML)*_G.

Il Listing 7.3 riporta la pipeline di CI.

```
1  ---
2  name: "CI"
3
4  "on":
5    push:
6      branches: ["**"]
7    pull_request:
8      branches: ["develop"]
9    workflow_dispatch: {}
10
11 jobs:
12   run-tests:
13     name: "Run Tests"
14     runs-on: ["archlinux"]
15     strategy:
16       fail-fast: false
17
18     container:
19       image: "localhost/erpai/python-ci:3.12"
20
```

```
21  steps:
22    - name: "Checkout"
23      uses: actions/checkout@v4
24
25    - name: "Set up Python"
26      run: python --version
27
28    - name: "Cache pip"
29      uses: actions/cache@v4
30      with:
31        path: "~/.cache/pip"
32        key: >-
33          ${ runner.os }}-pip-py312-${
34            ↪ hashFiles('**/requirements.txt') }}
35        restore-keys: |
36          ${ runner.os }}-pip-py312-
37          ${ runner.os }}-pip-
38
39    - name: "Install dependencies"
40      run: |
41        python -m pip install --upgrade pip
42        if [ -f auth-server/requirements.txt ]; then python -m pip
43          ↪ install -r auth-server/requirements.txt; fi
44        if [ -f duckdb/requirements.txt ]; then python -m pip
45          ↪ install -r duckdb/requirements.txt; fi
46        if [ -f forecasting/requirements.txt ]; then python -m pip
47          ↪ install -r forecasting/requirements.txt; fi
48        python -m pip install pytest pytest-asyncio anyio pytest-cov
49
50    - name: "Run tests with coverage"
51      env:
52        DATABASE_URL: "sqlite://"
53        DUCKDB_DATABASE_URL: "sqlite:///memory:"
54      run: |
```

51

```
python -m pytest -q --cov-report=xml --cov-report=html
```

Codice 7.3: Pipeline di Continuous Integration

Capitolo 8

Conclusioni

In questo capitolo vengono riassunti i risultati ottenuti durante lo stage, valutando il raggiungimento degli obiettivi prefissati e le conoscenze acquisite.

8.1 Copertura dei requisiti

La tabella [8.1](#) mostra la copertura dei requisiti elencati nella Sezione [4.2.2](#) (Tabella [4.2.2](#)): per ciascun requisito è indicato se è stato soddisfatto o meno.

Identificativo	Soddisfatto
R.O.0	Sì
R.O.1	Sì
R.O.2	Sì
R.O.3	Sì
R.O.4	Sì
R.O.5	Sì
R.O.6	Sì
R.O.7	Sì

Continua...

Continua...

Identificativo	Soddisfatto
R.O.8	Sì
R.D.0	Sì
R.D.1	Sì
R.D.2	Sì
R.D.3	No
R.D.4	No
R.F.0	No
R.F.1	No
R.F.2	Sì

Tabella 8.1: Tracciamento del soddisfacimento dei requisiti

8.2 Analisi del prodotto

Il prodotto sviluppato è già in uso presso il cliente e ora sta passando una fase di monitoraggio e raccolta *feedback* per futuri miglioramenti. Di seguito sono riportati i risultati ottenuti fino alla data di fine stage.

8.2.1 Previsioni annuali

Le previsioni hanno raggiunto un grado soddisfacente sull'andamento generale annuale: il $WAP\mathcal{E}_G$ medio è rimasto sotto il 15%, con una buona coerenza tra stagionalità attese e valori stimati. La dashboard di monitoraggio mostra l'allineamento tra curva storica e serie prevista, evidenziando scarti contenuti nei mesi di picco. La Figura 8.1 illustra l'andamento storico a confronto con le previsioni annuali, mentre la Figura 8.2 riporta il dettaglio numerico delle stime per ciascun periodo. La tabella mette affiancati valore storico, previsione e scostamento percentuale mese per mese, così da evidenziare dove il modello sovrastima o sottostima.

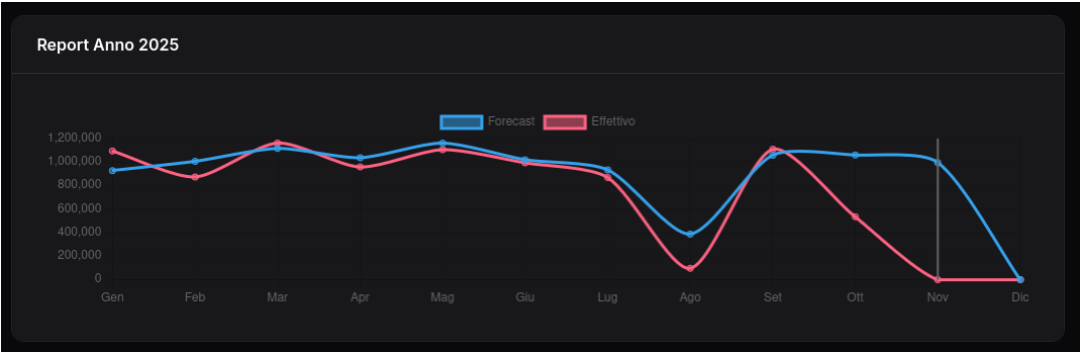


Figura 8.1: Andamento storico vs previsioni annuali (screenshot della dashboard)

Sales

2025

Ordini V

Mese	Forecast (€)	MAPE %	Effettivo (€)	Δ %
Gennaio	927.955,09	7.67 %	1.094.429,79	15.21%
Febbraio	1.006.220,29	7.67 %	874.196,59	15.10%
Marzo	1.115.381,39	7.67 %	1.162.138,80	4.02%
Aprile	1.036.838,20	7.67 %	958.463,56	8.18%
Maggio	1.160.798,11	7.67 %	1.105.589,22	4.99%
Giugno	1.019.991,35	7.67 %	993.924,08	2.62%
Luglio	933.278,23	7.67 %	869.494,76	7.34%
Agosto	388.129,72	7.67 %	97.598,26	297.68%
Settembre	1.058.363,19	7.67 %	1.108.955,40	4.56%
Ottobre	1.059.607,72	7.67 %	535.143,63	98.00%
Novembre	996.347,72	7.67 %	0,00	
Dicembre	0,00		0,00	

Figura 8.2: Tabella di dettaglio delle previsioni annuali

8.2.2 Feature più influenti

L'analisi SHAP mette in evidenza le variabili che guidano maggiormente le previsioni. Nel grafico, ogni punto rappresenta un campione: i colori indicano il

valore della feature (rosso = valore alto, blu = valore basso), mentre la posizione sull'asse orizzontale misura il contributo SHAP (a destra aumenta la previsione, a sinistra la riduce). La densità dei punti lungo l'asse mostra la distribuzione degli impatti. Alcune delle feature più influenti sono:

- **quantity_roll_med6**: media mobile a 6 periodi sulle quantità vendute; valori alti spostano nettamente a destra, segno che volumi recenti elevati trainano la domanda attesa.
- **article_avg_monthly_sale**: media mensile storica per articolo; contribuisce positivamente quando sopra la media, stabilizzando le previsioni su trend consolidati.
- **sales_history_ratio**: rapporto tra vendite recenti e storiche; valori alti indicano accelerazione e portano contributi positivi, mentre valori bassi riducono la stima.
- **summer_shutdown**: flag di chiusura estiva (es. agosto); i punti rossi a sinistra mostrano che quando la chiusura è attiva la previsione cala, come atteso per lo stop produttivo.

La Figura 8.3 visualizza la distribuzione dei contributi SHAP per le feature principali e aiuta a interpretare il segno e l'intensità dell'impatto sulle previsioni.

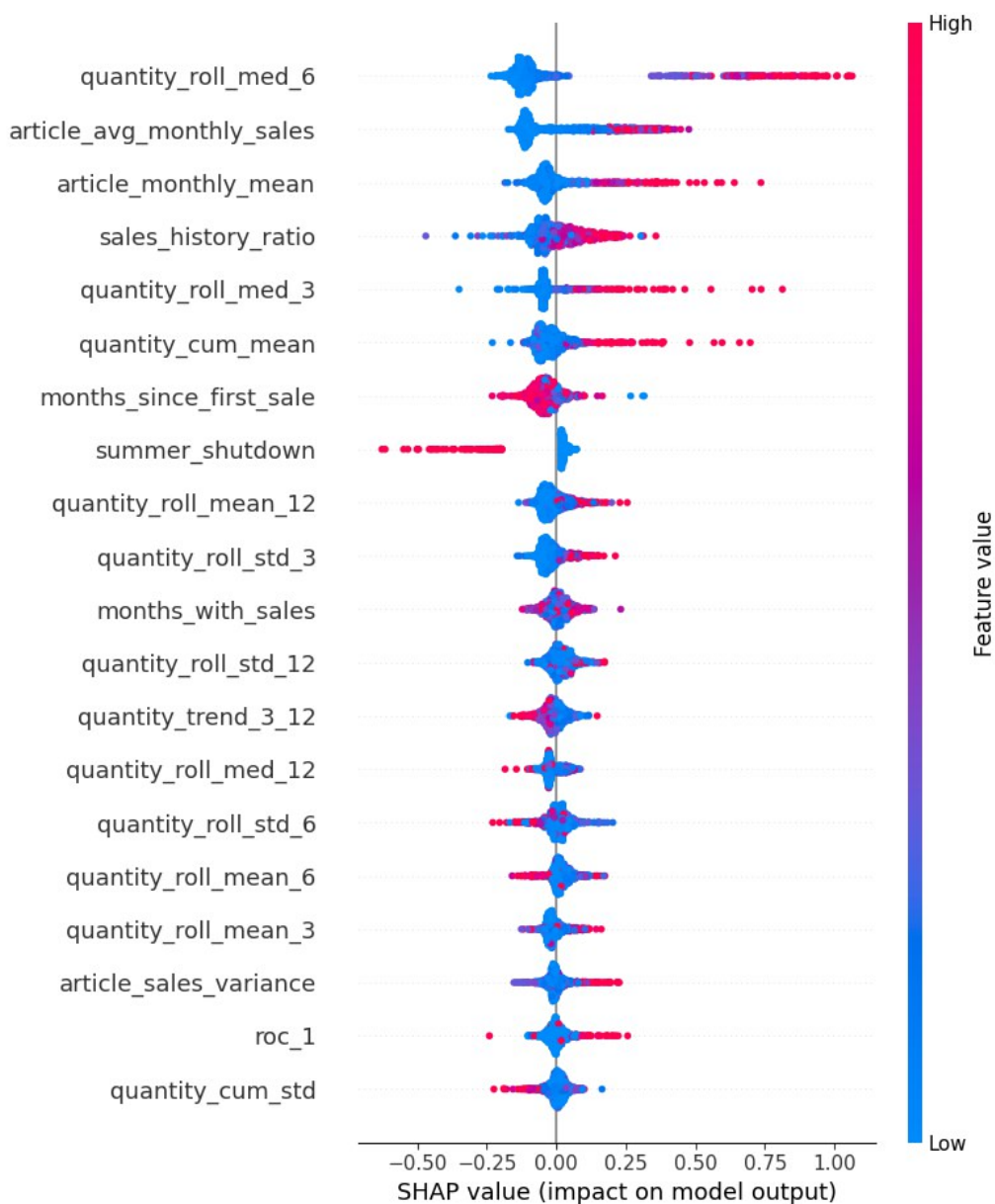


Figura 8.3: Feature più influenti secondo SHAP (valori e segno del contributo)

8.2.3 Miglioramenti futuri

Nonostante i risultati positivi, ci sono margini di miglioramento per affrontare alcune criticità emerse:

- **Previsioni per singoli articoli:** per quanto l'andamento generale sia buono, le previsioni a livello di singolo articolo mostrano scostamenti più marcati soprattutto per articoli con bassa storicità o stagionalità irregolare. Si prevede di integrare modelli specifici per cluster di articoli simili.

- **Automazione scelta iperparametri:** attualmente l'ottimizzazione degli iperparametri del modello XGBoost è manuale e richiede tempo. L'implementazione di tecniche di AutoML potrebbe velocizzare questo processo e migliorare le performance.
- **Automazione scelta feature:** l'attuale selezione delle feature si basa su analisi statistiche preliminari. L'adozione di metodi automatici di feature selection potrebbe identificare variabili più rilevanti e ridurre il rumore nei dati.

8.3 Valutazione personale

Durante lo stage ho avuto l'opportunità di applicare e approfondire diverse competenze tecniche acquisite durante il percorso universitario, in particolare nell'ambito del machine learning, della gestione di progetti software e dell'implementazione di pratiche *DevOps_G* come il testing automatizzato e la continuous integration. Ho imparato a lavorare in un contesto professionale, collaborando con un team eterogeneo e affrontando sfide reali legate alla gestione dei dati e alla scalabilità delle soluzioni. Questo stage ha rappresentato un'importante esperienza di crescita personale e professionale, che mi ha permesso di consolidare le mie conoscenze e di acquisire nuove competenze utili per il mio futuro lavorativo.

Per quanto riguarda il progetto sviluppato, ritengo che abbia raggiunto gli obiettivi prefissati, fornendo una soluzione efficace per la previsione della domanda e contribuendo al miglioramento dei processi decisionali dell'azienda. Sono consapevole che ci sono ancora margini di miglioramento e che il progetto potrà evolversi ulteriormente in futuro, ma sono soddisfatto del lavoro svolto e dei risultati ottenuti fino ad ora.